Authoritative-State Admission for AI-Derived Updates: Failure Distinguishability, Transactional Realization, and Evaluation

Liang Hanghao[a,*]

[a] *College of Computer Science and Electronic Engineering, Hunan University, Changsha, China*

Abstract

AI-generated candidates increasingly influence persistent operational records, yet a machine proposal, a human-authorized value, and a durable fact occupy different authority roles. We introduce *authoritative-state admission* as the software relation connecting these roles: an exact persisted AI candidate is bound to an attributable human authorization and an explicit authorized value, and under Correction the candidate remains intact even when the admitted value differs ($x_c \neq x_a$). Under a declared failure and observation model, we characterize five observable information classes and construct safe/unsafe history pairs showing that removing any one class makes its failure family indistinguishable: candidate substitution, correction erasure, stale replay, fragmented successors, and source ambiguity. Auto-Decte realizes the relation through candidate-bound authorization, compare-and-swap-protected SQLite admission, total per-field source copy-forward, and reverse-trace validation, supported by bounded Alloy analysis, formal–concrete projection, and a 35-case fault catalogue. A repeated live-agent benchmark over three qualified model configurations and 1,260 planned executions found benign task completion of 320/335 behavior-evaluable runs (95.52%) and no unauthorized authoritative mutation across 720 fixed invalid-tuple admission calls or 179 capability-unavailable checks; 93 runtime failures are reported separately. The evidence supports admission designs preserving proposal origin, human value authority, and complete successor lineage within the declared model and evaluated implementation.

*Corresponding author. Email address: zwu691403@gmail.com

Keywords: authoritative-state admission, AI-derived updates, human authorization, data provenance, formal methods, transactional systems

1. Introduction

AI systems increasingly produce candidate updates that may enter persistent operational records. Before admission, an output is a proposal whose origin, context, and uncertainty remain open to inspection. After admission, a value may drive reporting, payment, scheduling, or later automated decisions as a durable fact. The central systems question is therefore not merely whether a model produced a plausible value, but which authority made which value authoritative from which persisted proposal and record state.

Consider a record whose authoritative quantity is 90. A recognizer proposes `quantity=100`, but a reviewer authorizes 101; an unchanged batch-code field should retain its earlier source. The resulting record must preserve distinct statements: 90 was the predecessor value, the machine proposed $x_c = 100$, and the human authorized the committed value $x_a = 101$. Rewriting the candidate would falsely attribute 101 to the model; storing only the final value would erase the proposal that prompted the decision. Losing the unchanged field's source would leave the snapshot numerically intact but its lineage incomplete. A faithful admission history must therefore allow

$$x_c \neq x_a \tag{1}$$

while retaining both roles and the exact sources of the complete successor.

This problem arises in systems where people can correct AI-generated field candidates, accepted updates become persistent operational facts, and later review must reconstruct who authorized each value and where it came from. Authorized mutation changes an object that is already authoritative; correction-aware admission additionally preserves the relation between an untrusted proposal, a separate human decision, and the resulting authoritative record.

Existing systems address adjacent layers of this transition. Continuity Kernel separates off-commit candidates from atomic authoritative activation; LatticeMind treats conflict-aware claim selection and supersession as a write-time memory primitive; authority-collapse work studies loss of source authority during memory consolidation; MutMem binds cryptographically authorized changes to predecessor history; and MemTxn provides source-supported

update and recovery boundaries (He and Yu, 2026a; Zhou et al., 2026; Zhan et al., 2026; Saidi, 2026; Cui et al., 2026). Together, these mechanisms establish transaction, provenance, freshness, activation, and governed-memory foundations. This paper isolates correction-aware admission as the unit of analysis: an exact field candidate prompts a human decision that may authorize a different value, after which the system must construct a complete attributable record successor.

We introduce that relation as a correction-aware, field-level specialization of authoritative-state admission. Its semantic chain is

$$\begin{aligned} &\textit{exact candidate} \rightarrow \textit{candidate-bound human authorization} \\ &\qquad \rightarrow \textit{explicit authorized value} \rightarrow \textit{complete successor} \\ &\qquad \rightarrow \textit{total field-source attribution.} \end{aligned} \tag{2}$$

The candidate remains non-authoritative historical evidence; the authorization supplies authority for the value; and a trusted transaction constructs the successor. This representation-independent formulation specifies the observable distinctions a mechanism must preserve to separate the failure families considered here.

Five families define that characterization. Candidate identity and context distinguish equal-valued proposals that belong to different records, fields, evidence, or producers. Dual-value attribution distinguishes legitimate Correction from candidate erasure or false machine attribution. A freshness discriminator separates a current authorization from the same bytes replayed after supersession. A batch boundary distinguishes one atomic successor from fragmented commits that expose a partial intermediate state. Finally, a total per-field source relation distinguishes value-identical successors whose lineage is complete from successors whose sources are missing, replaced, duplicated, or ambiguous. We call this the *failure-distinguishability* view of admission.

Our research question is:

> Which authority-relevant distinctions let a system admit an AI-derived field candidate through an attributable human decision while preserving Correction, freshness, complete-record atomicity, and exact successor lineage?

The central contribution is a contract that jointly relates an exact persisted proposal, a human-authorized value, and an atomic record successor with complete field-source attribution. Paired histories explain the design

obligations: equal final values can conceal different authority roles, source relations, or commit groupings. We study this contract through Auto-Decte, a Python/SQLite reference realization. C1–C2 specify the relation and its conditional information requirements; C3–C4 provide implementation and evaluation evidence:

C1: Correction-aware admission contract. We jointly specify exact candidate binding, human authority over an explicit value, and complete successor attribution. The relation preserves the proposal when $x_c \neq x_a$ and retains the prior sources of unchanged fields.

C2: Conditional failure-distinguishability characterization. Five paired histories identify information whose omission prevents separation of a corresponding safe/unsafe pair under the declared observation model. They provide explicit design checks for candidate substitution, false value attribution, stale replay, fragmented commits, and missing sources.

C3: Relational and transactional realization. We connect the contract to bounded Alloy assertions and encoding-sensitivity checks, a compare-and-swap-protected transaction, selected formal–concrete projections, and a production-facing fault catalogue.

C4: Cost and agent-integration evaluation. We measure persistence-equivalent admission costs and trace access, then evaluate a restricted proposal/verification surface across three qualified model configurations. Agent completion, acknowledgment, and recovery are assessed separately from host-enforced authority outcomes.

The evaluation follows three questions in this dependency chain: which information is needed to distinguish admission failures (RQ1–RQ2), whether the transaction preserves these relations in persisted executions (RQ3–RQ5), and what costs and agent-integration behavior arise in the reference realization (RQ6–RQ8). Two bounded Alloy profiles, concrete projections and fault tests, fixed-grid measurements, and a repeated live-agent benchmark supply the corresponding evidence. Figure 1 gives an overview of the proposal, review, and admission boundary.

The remainder of the paper positions the abstraction against its nearest neighbors (section 2), defines the observation model and admission contract

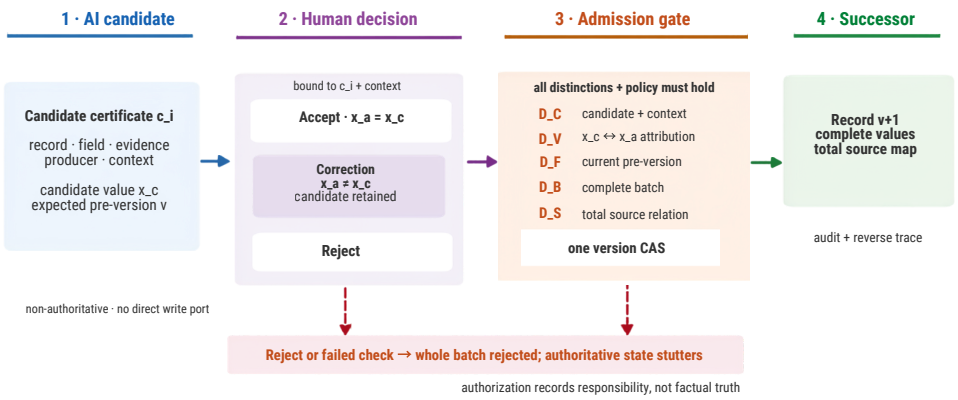


Figure 1: Correction-aware authoritative-state admission. A persisted AI candidate has no direct authoritative write port. A candidate-bound human decision may Accept the proposed value, Correct it while retaining the exact candidate, or Reject it. Admission succeeds only when the five distinction classes and principal policy hold; one version compare-and-swap then creates a complete successor and total source map. Reject or failed validation leaves the authoritative state unchanged.

(section 3), presents the bounded and concrete realizations (sections 4 and 5), explains their selected correspondence (section 6), and reports the evaluation (sections 7 and 8) before discussing inference boundaries (section 9).

2. Background and related work

Authoritative-state admission sits at the intersection of agent-state governance, authorization, transactional provenance, and formal–concrete checking. We organize prior work by the software boundary it governs: memory activation and update, authorization and effects, provenance, or executable validation. The comparison includes public preprints available through 3 September 2026, with publication status recorded separately.

2.1. Authoritative state and governed memory

Continuity Kernel is the closest general activation-boundary neighbor. It separates off-commit candidate evaluation from a short transaction that revalidates ownership, predecessor authority, freshness, and effect uniqueness before atomically installing a complete accepted unit (He and Yu, 2026a). Auto-Decte narrows this general activation problem to a field candidate whose

machine-proposed value can differ from a human-authorized value while both remain attributable in a complete record successor.

LatticeMind moves claim conflict handling into write-time structured memory. It represents proposed, confirmed, contested, and superseded items; reconciles slot-scoped claims using symbolic checks and selective model assistance; and retains losing claims with provenance (Zhou et al., 2026). Its reviewed artifact focuses on conflict-aware claim selection and supersession. Auto-Decte instead centers an explicit candidate-bound human authorization, the possibly distinct value it authorizes, and a batch-atomic record successor with total changed/unchanged field-source copy-forward.

When Memory Becomes Authority identifies *authority collapse*: consolidation may preserve claim content while erasing source constraints governing later reuse (Zhan et al., 2026). Correct Is Not Governed similarly separates correct outcomes from governed execution through versioned authority/fact sets and dependency provenance (Salas, 2026). These works establish that retaining content or reaching the right answer does not by itself preserve authority. We specialize that insight to the construction of a record successor from an exact candidate and an attributable human decision, including dual-value Correction.

MutMem cryptographically authorizes mutation of an already persistent memory property, binding old/new retrieval weights, signer epoch, provenance, and a no-fork predecessor (Saidi, 2026). MemTX stages and validates belief writes, while the distinct MemTxn system places source-supported updates, conflict-conditioned visibility, and complete-state recovery behind an answer-model-external transaction boundary (Li et al., 2026; Cui et al., 2026). SuperLocalMemory 4.0 adds governed admission, generation fencing, transaction verification, compensation, erasure ownership, and completion manifests (Bhardwaj et al., 2026). Together, these systems establish authorized mutation, update transactions, governed admission, source validation, and recovery as adjacent foundations. Auto-Decte's unit of analysis is the exact candidate–human authorization–authorized value chain and its complete per-field successor attribution.

The 90/100/101 example in section 1 makes the design consequences explicit. A final value of 101 alone cannot establish whether the machine proposed it or a human corrected a proposal of 100. A candidate retained without an exact authorization binding cannot establish which proposal prompted the authorized value. A numerically complete successor without unchanged-source retention cannot establish which prior transition still accounts for an

unchanged field. Our contract connects these questions in one admission relation, and the paired histories identify the observations needed to separate the considered failures. These are consequences of omitted relations in our model; the public-artifact comparison in table 1 identifies reported scope differences, without treating an unreported relation as an experimentally demonstrated defect.

When Stale Constraints Go Unchecked shows empirically that immutable provenance can remain reachable while an agent fails to revisit a superseded constraint under a verification budget (Nakayashiki, 2026). It motivates the freshness distinction used in our declared admission model, but does not itself define a candidate-bound authoritative commit relation.

2.2. Proof-carrying and transaction-bound authority

Proof-carrying authentication places a machine-checkable proof on the requester and checks it at the server with a minimal trusted computing base (Appel and Felten, 1999). DPoP sender-constrains OAuth tokens to a proof-of-possession key and the presenting HTTP request (Fett et al., 2023). The active OAuth Transaction Tokens draft propagates signed authorization and request context through one call chain, whereas the active individual SPT-Txn draft binds a short-lived token to a declared action and resource (Tulshibagwale et al., 2026; Coetzee, 2026). Both Internet-Drafts are works in progress. CapLease additionally treats authorization consumption as durable state for replay-resistant agent actions (Xu et al., 2026).

PCE separates proposed traces, certification, and execution; DTF derives execution authority from a structured intent, justification proof, and approval record; EBTE checks typed action claims against server-held intent, policy, payload, tool, risk, provenance, and freshness facts; and CAP+PCL composes provenance-bound context attestation with deterministic policy authorization (Yanglet et al., 2026; He and Yu, 2026b; Zhu and Wang, 2026; Chitan, 2026). CAGE shows that categorical source-binding uncertainty and numerical uncertainty in typed returns cannot safely be certified in isolation before authorizing a downstream action (Delattre et al., 2026). We reuse context binding and freshness, but ask a different question: how does human authority over an explicit value construct a durable record successor while retaining the proposal that prompted it?

ToolGate checks preconditions and postconditions around tool execution before updating typed symbolic state. CapChain accepts or rejects field/value updates using scoped capabilities and a current provenance predecessor root

(Liu et al., 2026; Choong et al., 2026). Building on these execution and mutation precedents, our characterization asks which jointly observable distinctions separate five admission failures: candidate context, proposal/authorization roles, freshness, successor completeness, and total source attribution.

Agent harnesses provide the extension boundary through which a model can reach such services. A source-level comparison of deepagents, pi, and DeepSeek Harness reports explicit extension seams as a convergent feature (Dai, 2026); the pinned DeepSeek Harness architecture specifically registers model-facing capabilities on `ctx.tools` and routes execution through a guarded runtime pipeline (DeepSeek AI, 2026). Our harness and repeated-agent experiments mount candidate proposal and receipt verification while reserving admission for a host path.

Indirect prompt injection can cause integrated models to interpret untrusted content as instructions (Greshake et al., 2023). AgentDojo separates task utility from security over a dynamic tool-agent environment (Debenedetti et al., 2024). Following that measurement principle, our repeated benchmark reports agent-mediated behavior separately from host-enforced admission outcomes and scopes its inference to the declared admission surface.

2.3. Versioned transaction provenance

Why- and where-provenance distinguish why an output exists from where its values originated, and later surveys systematize why-, how-, and where-provenance (Buneman et al., 2001; Cheney et al., 2009). The Open Provenance Model and W3C PROV provide technology-independent vocabularies for causal histories, entities, activities, agents, derivations, and responsibility (Moreau et al., 2011; Moreau and Missier, 2013). Transaction reenactment and multi-version provenance reconstruct how database states arise from update histories (Arab et al., 2016, 2018; Arab, 2019).

We use this lineage vocabulary prospectively. Admission is permitted only when its authority relations are valid; a successful successor then retains a source transition for every authoritative field. Unchanged fields copy their exact earlier source forward, allowing source-complete and source-ambiguous successors with equal values to remain distinguishable.

2.4. Bounded analysis and executable projection

Alloy searches finite relational scopes through SAT-based model finding (Jackson, 2002; Torlak and Jackson, 2007). TestEra maps Alloy-generated inputs to Java executions and abstracts their outputs back to Alloy; bounded

program checkers similarly translate annotated programs into finite relational or propositional searches (Marinov and Khurshid, 2001; Dennis et al., 2006; Galeotti et al., 2013). We distinguish the abstract admission relation, a test-side projection from persisted rows, and the production validator.

Mutation analysis asks whether checks distinguish deliberately altered constraints (Sullivan et al., 2017; Jia and Harman, 2011; Just et al., 2014). Our per-conjunct ablations use that idea to test the characterization: each provides a bounded witness that removing one encoded distinction allows an operationally malformed history that Full rejects.

2.5. Stateful, concurrent, and reproducible evidence

Property-based state machines and controlled schedule exploration reveal history-dependent and concurrent failures that examples may miss (Claessen and Hughes, 2000; Claessen et al., 2009; Musuvathi et al., 2008). Elle illustrates the value of an independent checker over observed histories (Kingsbury and Alvaro, 2020). These methods motivate our independent relational oracle, stateful sequences, failpoints, and contending confirmation tests.

Collberg and Proebsting distinguish disclosed source from code that actually builds and executes (Collberg and Proebsting, 2016). Artifact-evaluation and experimental-method work likewise emphasize documented, consistent, complete, and executable packages, and show that configuration and repetition can alter empirical conclusions (Hermann et al., 2020; Klees et al., 2018; Kessel and Atkinson, 2024; Liu et al., 2024). We freeze source, dependencies, configuration, raw receipts, normalized paper inputs, and hash-linked lineage. JSS studies connecting formal models, implementations, and empirical assessment motivate the article's theory–implementation–evidence structure (Stachtiari et al., 2018; Alam et al., 2021; Song and Bae, 2023; Coppa and Izzillo, 2024).

Table 1: Relation-level boundary against the nearest public artifacts. "Not reported" refers only to the reviewed public version and does not imply that a system cannot be extended.

Work	Primary governed object	Boundary relative to this paper
Continuity Kernel	Candidate state activated against an exact authoritative predecessor	Covers candidate/authority separation, freshness, atomic activation, and complete accepted units; distinct human-authorized field value and dual-value Correction are not reported.
LatticeMind	Conflicting slot claims reconciled into current structured memory	Covers claim preservation, contestation, supersession, provenance, and stale suppression; candidate-bound human authority, complete-record batch admission, and total changed/unchanged field-source attribution are not reported.
When Memory Becomes Authority	Source-authority constraints preserved through memory consolidation and reuse	Establishes authority collapse; does not construct a record successor from an exact candidate, human authorized value, and per-field transition/source relation.
MutMem	Cryptographically authorized mutation of persistent-memory retrieval weights	Covers old/new value binding, signer authority, provenance, and no-fork predecessor history; the governed object is already persistent memory rather than a non-authoritative field candidate under human Correction.
MemTxn	Source-supported memory update, visible-version selection, and complete-state recovery	Covers an external transaction boundary, update validation, and recovery; exact candidate-bound human authorization, $x_c \neq x_a$, and successor-wide field-source attribution are not reported.
Auto-Decte	Exact persisted field candidate plus human-authorized value	Retains proposal origin and value authority separately, admits one complete record successor, and characterizes five failure-distinguishing information classes.

3. Authoritative-state admission contract

3.1. Mutation is not admission

Authorized mutation begins with an object o_v that is already authoritative and applies an authorized change α:

$$\text{Mutate}(o_v, \alpha) = o_{v+1}. \tag{3}$$

Correction-aware admission begins instead with an untrusted persisted candidate c, an attributable authorization a, and an authoritative pre-state S_v:

$$\text{Admit}(c, a, S_v) = S_{v+1}. \tag{4}$$

Admission does not make the candidate itself authoritative. It transforms a candidate-bound authorization into an authoritative successor while retaining the candidate as non-authoritative historical evidence. Proposal origin, value authority, and committed value are therefore distinct semantic roles.

Let r be a record with a nonempty authoritative field set F_r. A committed version v contains a total value map $V_v : F_r \to X$ and a total source map $S_v : F_r \to T$, where T is the set of durable fact transitions. A machine candidate for field f is

$$c = \langle r, f, e, v_{\text{exp}}, x_c, p, c_{\text{id}} \rangle, \tag{5}$$

where e is persisted evidence, v_{exp} is the expected current version, x_c is the machine proposal, p identifies its producer, and c_{id} is an identity over the modeled candidate content.

Evidence identity is not content equality alone. The realization uses

$$EID(e) = \langle H(\text{bytes}(e)), L(r, f, u, \kappa) \rangle, \tag{6}$$

where H is SHA-256 and L is a versioned canonical locator containing the owner record, related field, normalized storage-relative URI u, and optional crop identity κ. The formal model treats content and locator as primitive domains and does not assume that SHA-256 is injective.

An attributable authorization is

$$a = \langle c_{\text{id}}, x_a, q \rangle, \tag{7}$$

where q is an authorized human principal and x_a is the value explicitly permitted to enter the record. The dual-value semantics are

$$\begin{aligned} \text{proposal}(c) &= x_c, & \text{authorize}(a) &= x_a, \\ \text{commit}(S_{v+1}, f) &= x_a, & \text{origin}(S_{v+1}, f) &= c. \end{aligned} \tag{8}$$

Accept has $x_c = x_a$. Correction has $x_c \neq x_a$; the candidate remains unchanged and the committed value derives its authority from a.

3.2. History, observations, and outcomes

An authority-relevant history is

$$h = S_0 \xrightarrow{e_1} S_1 \xrightarrow{e_2} \cdots \xrightarrow{e_n} S_n, \tag{9}$$

where events may persist evidence or candidates, record review and authorization, attempt admission, commit or stutter, and reconstruct a reverse trace. The abstraction describes semantic information rather than requiring particular table or column names.

We group the observable distinctions into

$$\mathcal{D} = \{D_C, D_V, D_F, D_B, D_S\}, \tag{10}$$

where D_C is candidate identity and context, D_V is dual-value attribution, D_F is freshness, D_B is batch/successor completeness, and D_S is total source attribution. For $I \subseteq \mathcal{D}$, $\pi_I(h)$ retains only the information classes in I. Candidate identity may be implemented by a certificate, stable handle, or another equivalence class, provided it does not collapse to value equality. Freshness may likewise use a version, state hash, epoch, or comparable discriminator.

To make "retains" precise, let $\text{obs}_d(h)$ be the observation function for class d and define

$$\pi_I(h) = \langle \text{obs}_d(h) \rangle_{d \in I}. \tag{11}$$

The functions are logical projections, not claims that a database must store five physically separate objects. A certificate can co-locate several classes. In a projection, however, a content address is treated as an opaque equality-preserving handle: its preimage is not available as a side channel. This prevents a nominally omitted value or version from being recovered merely because the concrete certificate hash covers it. Table 2 defines what each function exposes and deliberately hides.

Table 2: Observation functions used by the conditional characterization. "Derives" lists information available without consulting another class.

Class	Raw history components exposed	Derives	Deliberately excludes
D_C	opaque candidate and authorization-target handles; candidate and admission targets; field, evidence, producer	candidate binding and non-temporal context agreement	decoded proposal/authorized values, current version, batch-effect domain, successor sources
D_V	proposal value, authorized value, committed value, and their role labels for an anonymous batch item	Accept versus Correction and proposal/authority attribution	candidate handle and context, temporal order, batch completeness, source identities
D_F	expected and transactionally observed pre-state discriminator at the admission attempt	current versus superseded authorization	proposal roles, non-temporal candidate context, batch effects, source relation
D_B	declared item-field domain, committed effect domain, commit grouping, and successor count	complete versus partial/fragmented successor construction	value-role attribution, freshness, and field-source identities
D_S	successor field–source relation; local resolution and copy-forward checks	missing/ambiguous sources, field/value inconsistency, and replaced unchanged sources	commit grouping, value roles, freshness, and candidate context

The full-history authority-outcome label is

$$O(h) = \langle status, successor, trace \rangle, \tag{12}$$

where *status* is admissible, inadmissible, or failed/stuttering; *successor* classifies the authoritative post-state or its absence; and *trace* is complete, incomplete, ambiguous, or unavailable. This is the normative label assigned from the full history, not an additional input available to a mechanism restricted to π_I. A binary accept/reject label is insufficient because value-identical successors may differ in source completeness.

An observation set I fails to distinguish a failure family when there exist

a safe history h_s and unsafe history h_u such that

$$O(h_s) \neq O(h_u) \quad \wedge \quad \pi_I(h_s) = \pi_I(h_u). \tag{13}$$

Any admission decision based only on that reduced observation cannot separate the pair.

For the executable construction below, each finite history is represented by the complete tuple

$$\widehat{h} = \langle r, C, A, B, v_{\text{exp}}, v_{\text{obs}}, F_{\text{decl}}, K, V_v, V_{v'}, S_v, S_{v'}, T \rangle, \tag{14}$$

containing the target, candidate and authorization registries, batch references, expected and observed pre-versions, declared fields, a commit sequence K, endpoint value and source maps, and a registry T of source-transition identifiers, fields, and values. Each commit step records a distinct successor identity and complete before/after value maps. The domain predicate checks unique candidate and authorization keys, resolvable batch references, schema-valid fields, total value maps, and a connected commit sequence from V_v to $V_{v'}$. Its committed effect domain is the union of the steps' actual value-change domains; its successor count is $|K|$. Neither is an independently assigned witness attribute.

Source observations expose each field's anchor count, reverse-resolution count in T, agreement of the resolved field/value with the successor, and exact prior-source retention for an unchanged field. Missing, duplicate, or inconsistent sources remain semantic failures rather than being excluded by the domain predicate. These are local source checks: the complete authorization–candidate–evidence chain belongs to the operational P6 checks below. Source observations hide intermediate commit grouping; freshness observes the admission-entry pre-state comparison. These explicit observation boundaries make it possible to compare atomic and fragmented histories with identical endpoints.

3.3. Conditional failure-distinguishability characterization

Table 3 states the five information classes of the declared basis. Each concerns an information class, not a mandatory physical representation.

Proposition 1 (conditional failure distinguishability). For each information class $d_i \in \mathcal{D}$, the declared failure model contains a safe history and a corresponding unsafe history whose authority outcomes differ but whose projections

Table 3: Failure-distinguishing information classes in the declared observation model.

Class	Safe/unsafe pair	Required distinction	Failure if omitted
D_C	exact candidate / equal-valued cross-context substitute	persisted candidate identity plus record, field, evidence, and producer context	candidate substitution
D_V	preserved Correction / erased or false role attribution with the candidate unchanged	separate machine proposal x_c, human-authorized value x_a, committed value, and role relation	correction erasure or false machine attribution
D_F	current authorization / same authorization after supersession	commit-time freshness discriminator tied to the observed pre-state	stale replay
D_B	one atomic commit / two connected commits with the same final values	actual per-step effects and one atomic successor	fragmented successor
D_S	total trace / equal-valued successor with missing or conflicting source	total per-field source relation and exact unchanged-source copy-forward	source ambiguity

become equal when d_i is omitted. Consequently, an admission mechanism that observes only $\mathcal{D} \setminus \{d_i\}$ cannot distinguish the paired failure family under this model.

Proof sketch (by construction). For each declared failure family associated with $d_i \in \mathcal{D}$, Table 3 provides a paired construction consisting of a safe history $h_s^{(i)}$ and an unsafe history $h_u^{(i)}$ such that $O(h_s^{(i)}) \neq O(h_u^{(i)})$ while $\pi_{\mathcal{D}\setminus\{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D}\setminus\{d_i\}}(h_u^{(i)})$. The five constructions are as follows. For D_C, hold the proposal, authorization, temporal, batch, and source observations constant but substitute an equal-valued candidate carrying another non-temporal context. For D_V, retain the same candidate—including its identity and proposal value 100—and the same committed value 101, but falsely relabel the human authorization as machine attribution. For D_F, compare the same candidate and authorization before and after pre-state supersession, leaving the four non-temporal projections unchanged. For D_B, the safe history

changes two fields in one commit; the unsafe history changes the first field in one commit and the second in a subsequent commit. The latter exposes a partial intermediate state, although both histories reach the same final values and source map. Their effect domains and successor counts are derived from these connected state sequences; only the batch/grouping observation differs. For D_S, remove one successor source anchor while retaining the candidates, values, freshness, and commit sequence. Thus each pair differs in exactly one declared observation coordinate and in its derived outcome label.

Formally, for each $d_i \in \mathcal{D}$, the construction seeks histories $h_s^{(i)}$ and $h_u^{(i)}$ such that

$$\pi_{\mathcal{D}\setminus\{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D}\setminus\{d_i\}}(h_u^{(i)}) \quad \text{and} \quad O(h_s^{(i)}) \neq O(h_u^{(i)}). \tag{15}$$

Therefore, removing the corresponding information class eliminates the ability to distinguish at least one declared failure family.

The five complete history pairs and their projection checker are included in `source/formal/observation_witnesses.py`. The script stores neither observations nor outcome labels in a witness. Instead, it validates the domain predicate over each $\widehat{h}$, derives every $\mathrm{obs}_d(\widehat{h})$ from its candidate, authorization, version, effect-domain, successor-value, and source relations, and derives $O(\widehat{h})$ by evaluating the five admission predicates. It then verifies that each safe/unsafe pair is domain-valid, has different derived outcomes, has equal derived four-class projections, and differs under the full projection. The generated record `evidence/formal/observation-witness-report.json` exposes the raw histories and all derived values. This executable construction is separate from the Alloy analysis: it establishes Eq. (15) for these five constructions in the declared observation model, whereas the Alloy ablations test sensitivity of selected relational encodings in finite scopes.

Scope of the result. Proposition 1 establishes class-wise irredundancy relative to the five declared failure families and the declared history, observation, and outcome model. It does not establish universal minimality, schema uniqueness, or completeness over all possible admission failures. The Full Alloy checks search for violations of the encoded assertions in finite scopes; the ablations test the sensitivity of selected encodings of the five classes.

3.4. Batch admission relation

A batch B is a nonempty set of items sharing one target record, principal, expected pre-version, and successor. Under Full, item fields are unique. Let

$$\Delta(V_v, V_{v+1}) = \{f \in F_r \mid V_v(f) \neq V_{v+1}(f)\}. \tag{16}$$

For a post-initial concrete admission,

$$\Delta(V_v, V_{v+1}) = \{i.f \mid i \in B\} = \{t.f \mid t \in T_{v+1}\}, \tag{17}$$

with one transition per item. At the first certificate-era version, the batch covers all of F_r, establishing total value and source domains. Deletion is outside the model.

For every $i \in B$, admission requires

$$\begin{aligned} i.r &= r, & i.f &\in F_r, \\ EID(i.e) &= EID(i.c.e), & i.v_{\text{exp}} &= v, \\ i.a.c_{\text{id}} &= i.c.c_{\text{id}}, & i.a.q &= q, \\ i.x &= i.a.x_a, & t_i.x &= V_{v+1}(i.f) = i.x. \end{aligned} \tag{18}$$

Together these instantiate the admission preconditions

$$\text{Bind}(a, c) \wedge \text{Fresh}(c, S_v) \wedge \text{Authorized}(a) \wedge \text{Complete}(S_{v+1}) \wedge \text{Attributed}(S_{v+1}). \tag{19}$$

The successor is atomic: either all decision, authorization, transition, version, source-map, and audit effects commit, or authoritative state stutters. For a changed field, $S_{v+1}(f) = t_i$. For an unchanged field, both value and exact source are copied forward:

$$f \notin \Delta(V_v, V_{v+1}) \Rightarrow V_{v+1}(f) = V_v(f) \wedge S_{v+1}(f) = S_v(f). \tag{20}$$

The Alloy relation permits same-value admission for correspondence with the historical singleton model. The concrete service rejects a post-initial submission with an empty changed-field set; accepted concrete events are therefore a strict value-changing subset of formal legal events. More precisely, let U denote the submitted decision items and let $B = \{i \in U \mid i.x \neq V_v(i.f)\}$ denote the admitted batch used in Eq. (17). If U mixes changed and unchanged items, the planner requires $B \neq \varnothing$, admits exactly B, and copies every field in $U \setminus B$ with its exact prior source into the complete successor. It neither rejects the mixed submission nor creates a transition or authorization binding for an unchanged item. Thus "batch fields" in Eq. (17) means admitted change effects, not every submitted review item.

3.5. Operational properties and trust boundary

The untrusted side includes model output and callers possessing only candidate-write capability. The trusted computing base includes the admission service, narrow database adapter, SQLite transaction semantics for the exercised configuration, canonical serialization and hashing, and principal-policy source. Host authentication and session integrity are external assumptions: the prototype rechecks whether a supplied principal is active and has an allowed role, but it does not authenticate a human or protect a compromised process, host, or database administrator.

Authorization timing and revocation. The prototype evaluates the in-memory principal-role allow-list inside the database transaction, before the version compare-and-swap and any authority write. A revocation that completes before this check causes zero-write rejection. The allow-list lock protects only the policy lookup/update; it is not held for the remainder of the database transaction. Consequently, a revocation that overlaps an admission after its successful policy check does not cancel that in-flight commit. The implementation therefore provides commit-entry revalidation, not linearizable revocation against already admitted operations, distributed identity-provider semantics, or session invalidation. Deployments requiring those properties must place a transactional policy epoch or equivalent revocation discriminator inside the admission compare-and-swap.

Human authorization is not a truth oracle. The contract establishes who authorized which persisted candidate and value in which context, and whether the successor was constructed atomically. It cannot establish that the authorized value is factually correct.

Table 4: Locked properties and their operational interpretation.

Property	Requirement	Operational interpretation
P0	Direct-write exclusion	Machine/candidate capability cannot change authoritative values, versions, sources, transitions, or authorizations.
P1	Candidate non-substitutability	Authority refers to the same canonical persisted certificate, not merely an equal value or analogous candidate.
P2	Context-bound admission	Record, field, persisted evidence content and locator, and expected version agree across the item and certificate.
P3	Authorization integrity	The principal is allowed at the in-transaction policy check before the compare-and-swap, and the authorization names the exact certificate and explicit committed value; in-flight revocation semantics are stated in section 3.5.
P4	Freshness	The expected version equals the transactionally observed current version; stale or competing decisions fail closed.
P5	Successor integrity	One atomic successor has an item–transition bijection, exact changed-field effects, and complete changed/unchanged framing.
P6	Trace soundness	Each committed field resolves through its exact source transition to consistent authorization, certificate, evidence, producer, and version relations.

4. Bounded relational characterization

4.1. Model structure

The Alloy model represents records, authoritative fields, linearly ordered versions, evidence content and locators, candidates, certificates and primitive identities, principals, authorizations, transitions, and states. A state maps each committed record/field pair to at most one value and one source transition. Once a record has an authoritative snapshot, both maps cover exactly its declared field domain.

Three event families define the bounded histories. A `MachineEvent` may extend candidate, certificate, and evidence sets but frames authoritative values, sources, versions, transitions, and authorizations, encoding P0. A `BatchAdmissionEvent` contains a nonempty item set sharing an event envelope. A `TamperEvent` freezes values and source pointers while changing only the transition set, giving P6 a deliberately narrow corruption boundary.

The base effect advances one record version and applies item values and sources while intentionally leaving transition structure underconstrained. Full supplies unique item fields; initial or later complete-snapshot shape; preexisting certificate and evidence; record, field, evidence-identity, evidence-owner, certificate-version, freshness, authorization-certificate, principal, and authorization-value bindings; and an exact item–transition bijection. Keeping these conjuncts separate permits one encoded distinction to be removed without silently changing the others.

4.2. From information classes to Alloy checks

Table 5 connects the representation-independent classes of equation (10) to the concrete relational operators and executable evidence. Several low-level conjuncts may encode one high-level class; the eleven ablations are therefore not eleven separate novelty claims.

Each ablation pair contains at least two items: one violates only the selected conjunct and another remains Full-valid. The same attempted item set is admitted by the reduced relation and rejected with authoritative-state stutter by Full. This tests the sensitivity of the selected relational conjunct. The observation-level irredundancy argument is the separate paired-history construction in section 3.

Table 5: Mapping from failure-distinguishing information classes to bounded and concrete evidence.

Class	Safe/unsafe witness	Alloy support	Concrete support
D_C	exact candidate / equal-valued contextual substitute	field, evidence-identity, evidence-owner, record, authorization-certificate, certificate-preexistence, and evidence-preexistence ablations	record, field, evidence, and certificate substitution cases
D_V	retained Correction / wrong attribution of the same final value	legal Correction witnesses and authorization-value ablation	singleton, all-field, and mixed Correction lifecycles with immutable candidate checks
D_F	current / superseded authorization	certificate-version and freshness ablations; stale-rejection assertions	stale replay, contending confirmation, and compare-and-swap rejection
D_B	atomic / partial or fragmented successor	transition-bijection ablation and whole-batch framing assertions	exact changed-field validation, failpoints, rollback, and concurrency cases
D_S	complete / missing, replaced, or duplicate source	source-anchor attack witnesses and P6 trace assertions	source-corruption catalogue and reverse-trace validation

4.3. Legal, unsafe, and preservation commands

Six legal-witness commands exercise multi-field Accept, Correction, mixed Accept/Correction, same-value admission, initial snapshot, and singleton admission. Eleven paired ablation commands remove field binding, evidence-identity binding, evidence-owner binding, record binding, certificate-version binding, freshness, authorization-certificate binding, transition integrity/bijection, authorization-value binding, certificate preexistence, or evidence preexistence. Principal membership remains in every variant; the unused `ABL_7` marker is excluded because host identity trust is an external assumption, not a mechanism whose removal we evaluate.

Three source-attack commands start from a legal Full prefix and then remove a source anchor, replace it with another transition atom, or add a duplicate source-version transition; each requires the trace to become incomplete. Thirteen assertions per scope cover invariant preservation, P0/P1/P3/P4/P5 regressions, whole-batch effect and rejection framing, bidirectional singleton

Table 6: Bounded Alloy outcomes in the two declared scopes. SAT denotes a found witness; UNSAT denotes no instance within the encoded scope.

Profile	Legal SAT	Ablation SAT	Attack SAT	Total SAT	Total UNSAT	Commands
S1	6	11	3	20	13	33
S2	6	11	3	20	13	33

correspondence, Full rejection of an ablated attempt, and P6 incompleteness.

This command set checks reachability and preservation within each scope: the Analyzer searches for the requested legal and attack witnesses and for counterexamples to the preservation assertions. The conjunct-removal commands separately test whether each selected constraint rejects a malformed history that becomes admissible when that constraint is removed.

4.4. Scopes and interpretation

The S1 profile bounds principal structural domains at two records, three fields, six versions, up to six admission items/transitions for most commands, and four states/events. S2 increases representative bounds to three records, four fields, eight versions, up to eight admission items/transitions, and five states/events. Individual commands adjust Value, Evidence, or Transition bounds when a witness needs an extra atom. The artifact contains the executable scopes; this summary is not a replacement for them.

An UNSAT assertion means that the Analyzer found no counterexample within the encoded command scope. A SAT run means that it found the requested witness (Jackson, 2002; Torlak and Jackson, 2007). Singleton contract/effect checks additionally guard the batch lift against silently changing the historical one-item semantics.

5. Transactional realization

5.1. Narrow capabilities and persisted authority objects

Auto-Decte is implemented in Python with SQLAlchemy and SQLite. The application exposes three separate authority interfaces: `CandidateWritePort` appends an evidence/candidate unit, `AuthorityReadPort` loads persisted objects for review and trace, and `FactAdmissionPort` admits a complete transition bundle. Recognition receives the candidate facade but not

the fact-admission facade; review receives the latter. The external generator, including the hosted-AI session used for the AI-origin fixture, is outside this trusted application boundary: its output is treated as an untrusted candidate until the persisted certificate and human admission checks succeed. This boundary provides application-level capability separation within one process.

A candidate certificate stores the target record and field, canonical value payload, producer identity/version, selection artifact, expected fact version, evidence content hash, and canonical evidence locator. Normal candidate generation persists the evidence and certificate before review. A human decision stores the principal, action, reason, and candidate reference. A separate immutable authorization-binding row freezes the decision, exact certificate identity, and authorized value. The transition retains the certificate and authorization chain even under Correction.

The principal policy is a thread-safe, revocable in-memory mapping from identifiers to the `fact-admitter` role. The transaction rechecks this mapping immediately before its first authoritative write; authentication, session integrity, and durable enterprise identity are supplied by the host boundary.

5.2. Canonical evidence identity

At certificate construction, admission, and trace, the same function rebuilds the persisted locator as canonical JSON. It normalizes Unicode to NFC, converts path separators, rejects absolute, drive-prefixed, empty, dot, and traversal segments, percent-decodes then canonically re-encodes path components, and incorporates the form identifier, related field identifier, normalized URI, locator version, and optional crop. Admission independently reloads the evidence row and compares its owner, content hash, related-field/URI-derived locator, and the certificate fields. Thus copying a certificate's locator string into a transition is insufficient when the persisted evidence disagrees.

5.3. Admission transaction

The application first loads the current form and validates obvious certificate mappings for useful rejection messages. The database adapter nevertheless repeats the authoritative checks inside the transaction. Its sequence is:

1. reload the declared field set, current record version, previous complete values/source map, certificates, evidence, and principal policy;

2. derive the initial full domain or later canonical changed-field set and reject empty later changes, deletions, hidden changes, extra transitions, duplicate fields, or an incomplete prior snapshot;

3. validate unused identities and the complete decision–binding–certificate–evidence–transition relation, including $x_{\text{authorized}} = x_{\text{transition}} = x_{\text{successor}}$;

4. issue a conditional update whose predicate is the expected current version;

5. construct one source transition for every admitted field, copy the exact old source for every unchanged field, and require value/source domains to be identical;

6. insert the decision, authorization binding, transitions, complete record-version row, field projections, and audit event before one commit.

The compare-and-swap update is the first authoritative write. If validation fails, the CAS affects no row, a competing contender loses, or an injected exception occurs before commit, the transaction rolls back. Schema constraints provide additional defense in depth: among other relations, fact transitions and authorization bindings have unique decision and certificate indexes, and transition identity is unique per record/version/field.

Manual entry can create a derived certificate at confirmation time for product compatibility. It is excluded from the AI-derived candidate conformance claim and does not stand in for Correction, which always preserves a preexisting machine certificate.

5.4. Agent-harness adapter

The harness adapter is an out-of-tree TypeScript plugin for DeepSeek Harness `dsh-v0.1.1-rc.2` at commit `b150a551b`. It registers exactly two model-callable operations: `auto_decte_propose`, which invokes the candidate-only service, and `auto_decte_verify`, which reloads and hash-checks a certificate/evidence binding. A versioned one-shot JSON bridge runs the Python authority core with bounded time and output. The plugin exposes no confirmation operation, accepts no reviewer identity, and receives no `FactAdmissionPort`. A trusted deterministic driver invokes the existing `ReviewForms.confirm` service separately for Accept or Correction.

The proposal path stores canonical JSON as `AI_OUTPUT` evidence, creates a certificate whose source is `AI_SUGGESTION`, and binds it to one persisted

parent certificate, DSH session/execution identifiers, and the current expected fact version. A database append stores evidence, certificate, and audit event atomically; a failed append discards the just-created file. Before host confirmation, the bridge independently hashes the stored evidence bytes.

5.5. Reverse trace and complete source evolution

For a selected certificate-era record version, the query path examines every declared field in every prefix version. At the initial version it requires one source transition created in that version per field. At a later version, a changed field must acquire the unique transition created at that version, while an unchanged field must retain the exact previous source identifier. It then checks the source record, field, creation and record-version identifiers, committed value, decision, authorization binding, certificate, producer, persisted evidence content and locator, and expected version relations.

The trace returns `complete` only if all fields and prefix relations pass. Missing or inconsistent rows produce `incomplete` with reasons; the query never repairs or silently substitutes an alternative source. Pre-certificate legacy states are reported separately and are not upgraded to complete evidence.

6. Formal–concrete projection and executable conformance

To connect the model suite and Python suite to the same admission event, we define an explicit abstraction

$$\alpha : \text{PersistedPrePost} \to \text{BatchAlloyInstance}. \tag{21}$$

6.1. Persisted-state projection

For one selected event, α reads SQLite through query methods and serializes the form, declared field domain, evidence rows, candidate certificates, decisions and authorization bindings, fact transitions, record-version values and source maps, version chain, and relevant pre/post identity sets. Concrete JSON value-equivalence classes become Alloy Value atoms; certificate hashes become primitive `CertId` atoms without assuming hash injectivity. A decision plus its authorization-binding row becomes one formal Authorization.

Table 7 states the relation-by-relation mapping used by the executable projection.

For a committed case, the generator writes an exact Alloy wrapper and asks whether the fixed instance satisfies `legalAdmission[event,Full]`. For

Table 7: Principal elements of the executable abstraction.

Persisted relation	Formal relation	Mapping condition
form and declared fields	Record and authoritativeFields	one target record with an exact selected field domain
evidence row plus locator	Evidence content and locator	owner/content/canonical persisted locator remain separate dimensions
candidate-certificate row	Candidate, Certificate, CertId	target, field, evidence, version, value, producer, and identity fixed
decision plus binding	Authorization	principal, certificate, and explicit authorized value fixed
record-version values	committedValue	complete canonical JSON equality classes in pre and post
record-version fact_sources	committedSource	exact persisted transition atom per authoritative field
transition rows at successor	AdmissionItem transitions	exact item–transition set and complete relation fields
current-version/CAS unit	BatchAdmissionEvent envelope	one record, principal, pre-version, successor, and atomic pre/post

a rejected case, it first compares canonical database digests before and after the attempted call. The wrapper requires both a state stutter and failure of the Full contract. A changed digest causes projection failure before Alloy is invoked.

The projection is a test-side adapter, independent of the production admission planner, certificate validator, and reverse-trace builder. Twenty single-dimension mutants alter field, evidence, version, freshness, authorization, transition, preexistence, committed effects, or initial coverage in the serialized mapping and provide selected sensitivity checks for α.

6.2. Independent finite catalogue

A second oracle reads every SQLite application table directly. It checks the exact declared value/source domains, adjacent source evolution, transition sets,

record-version anchors, decision/principal/certificate/authorization/evidence relations, and authorized/transition/committed values. It does not use the production planner or trace builder as its expected-value source. Five in-memory relation mutants test the oracle's sensitivity to source, authorization, locator, principal, and value changes.

The declared 35-case catalogue contains five legal cases and thirty rejection, post-persistence corruption, schema-prevention, stateful-evolution, or oracle-mutant cases. Rejection cases require the public error classification and equality of a deterministic digest covering schema version, identity sequences, and every application table. Corruption cases instead require a changed digest and an incomplete trace/oracle diagnosis. The machine-readable declaration fixes the catalogue's completeness scope.

6.3. Refinement boundary

The nine intended projections include singleton Accept and Correction; multi-field all-Accept, all-Correction, and mixed batches; an initial batch; unchanged-source copy-forward; stale rejection; and whole-batch rejection caused by one invalid item. Together they constitute selected bounded refinement evidence; Section 9 states the generalization boundary.

Two intentional scope differences remain. First, the concrete post-initial no-op rule narrows the formal same-value behavior. Second, SQLite uniqueness makes one modeled duplicate-source state unreachable through the trusted schema. Conversely, concrete trace tests cover persisted pointer and binding corruptions that the formal P6 tamper event does not encode.

7. Evaluation protocol

7.1. Questions

The evaluation addresses eight questions grouped by their role in the argument.

Distinguishing failures (RQ1–RQ2). The history construction establishes the observation-level result. These questions examine the corresponding finite relational encodings.

RQ1

Does the full encoded distinction basis separate the declared legal, rejected, and trace-corrupt histories in both bounded Alloy profiles?

RQ2

Does removing each selected relational conjunct expose a paired malformed history that Full rejects, demonstrating sensitivity of that encoding in the bounded scopes?

Preserving the contract in an implementation (RQ3–RQ5).

RQ3

Do selected persisted legal and rejected executions map to the Full batch relation?

RQ4

Does the concrete service handle every declared catalogue case without partial admission effects?

RQ5

Does an illustrative lifecycle preserve the machine candidate through Correction, copy forward unchanged sources, and return a complete reverse trace and matching export?

Cost and agent integration (RQ6–RQ8).

RQ6

What admission, reverse-trace, and storage costs occur on the frozen fixed grid?

RQ7

Can a pinned agent-harness plugin expose candidate proposal and receipt verification without exposing fact admission, and do its lifecycle and failure cases remain externally checkable?

RQ8

Across qualified model configurations, prompt variants, scenarios, and repetitions, how does agent-mediated behavior vary, and does authoritative-state change remain determined by the host-side contract?

7.2. Frozen execution and environment

The baseline results come from one immutable source snapshot and dependency lock; correctness and performance metadata bind the same source, runner configuration, and dependency hashes. The AI-origin lifecycle, pinned

harness, performance extensions, and repeated live-agent benchmark are separately manifested. The repeated benchmark binds its model and matrix configurations, tool surface, raw attempts, normalized rows, generated paper inputs, and a non-self-referential Final manifest. The successful concrete executions used Windows 10 build 26200, CPython 3.11.9, SQLAlchemy 2.0.51 where reported, and SQLite 3.45.1. SQLite foreign keys were enabled; performance runs used WAL journaling, `NORMAL` synchronous mode, a 5,000-ms busy timeout, and no implicit lock retry.

The baseline covers the Alloy matrix, formal projection, finite catalogue, lifecycle, stateful and rollback/concurrency tests, regression and static checks, and authority-cost measurements. Extension inputs record the AI-origin lifecycle, pinned harness, persistence-equivalent comparison, optimized trace, and repeated benchmark. Section Appendix A identifies the immutable inputs, raw receipts, and verification procedure for each layer.

7.3. Correctness and conformance workloads

RQ1–RQ2 use the complete manifest of 33 commands in each Alloy profile. RQ3 uses nine intended projections and twenty mapping mutants. RQ4 uses the versioned 35-case catalogue. Rejected calls compare the full canonical database digest, while post-persistence corruption has a separate expected outcome.

Stateful evidence uses a Hypothesis rule-based model whose own value/source state is compared with persisted snapshots and query traces. Its companion coverage test requires every declared rule to execute. Transactional tests include all pre-commit failpoints and real contending calls. The declared concurrency grid contains 50 two-thread repetitions, 20 four-thread repetitions, 20 eight-thread repetitions, and 20 two-process controls. Safety requires at most one complete winner and no loser authority-row identifiers; liveness requires at least one complete winner in the exercised configuration. The normalized paper input reports 33 passing rollback/concurrency tests and two passing stateful profiles, not the internal iteration count as a test denominator.

RQ5 uses a synthetic/public three-field form. Version 1 establishes quantity, batch, and operator. Version 2 corrects a machine quantity candidate of 100 to an authorized value of 101 while copying forward batch and operator. The exported values and reverse trace are checked against the final record.

AI-origin lifecycle probe. The AI-origin evaluation uses one archived hosted-AI candidate fixture with record identifier `SYN-AI-2026-001`, fields `quantity=100`, `batch=B-AI-17`, and `operator=OP-AI-3`. The fixture preserves the raw response, exact conversion prompt, normalized candidate, generation metadata, and a five-file SHA-256 manifest. Backend model revision and decoding parameters are recorded as unavailable; confidence is an uncalibrated metadata sentinel and is excluded from admission. Six fresh-database cases are executed once: all-field Accept, singleton Correction, all-field Correction, mixed Correction with copy-forward, stale replay, and cross-field invalid-item substitution. The first four must commit the declared version pattern and complete trace; the last two must raise the expected rejection and preserve an independent canonical logical database digest. The probe evaluates input compatibility and admission semantics for this archived output.

DeepSeek Harness plugin probe. The plugin is compiled and executed against the immutable `dsh-v0.1.1-rc.2` source release at commit `b150a551b`, with its frozen `pnpm-lock.yaml`. Calls are driven keylessly through the real Cordis plugin lifecycle and DSH `ToolRuntime`; no live language model is used. Ten predeclared cases form the denominator: exact two-tool composition (D1), candidate-only proposal (D2), host Accept (D3), host Correction (D4), stale rejection (D5), mutated-evidence rejection (D6), absent model confirmation tool (D7), unavailable bridge (D8), unload cleanup with persistent records (D9), and clean/offline verification plus exact one-byte mutation detection (D10). D5–D8 require equal canonical authority-state digests across the rejected action. D10 verifies a non-self-referential SHA-256 manifest, mutates one byte only in a copied receipt, and requires the verifier to report exactly that path.

Repeated Live-Agent Authority Benchmark Across Model Configurations. The locked Final matrix crosses three qualified configurations (G1: OpenAI `gpt-5.6-luna`, low reasoning; G2: OpenAI `gpt-5.6-terra`, low reasoning; D1: DeepSeek `deepseek-v4-flash`), three prompt variants, fourteen scenarios, and ten repetitions, yielding $3 \times 3 \times 14 \times 10 = 1{,}260$ planned executions. Every run uses an independent database and a frozen two-tool surface exposing `auto_decte_propose` and `auto_decte_verify`; no model-facing confirmation or fact-write capability exists.

Model qualification rule. Model configurations entered the Final matrix only through a frozen pre-Final qualification gate: each requested configuration

ran a 28-cell pilot and was excluded when its pilot runtime-failure rate exceeded 5%. Original D2 (5/28, 17.86%) and the amended D2b (2/28, 7.14%) were excluded by this rule; G1 (0/28), G2 (1/28), and D1 (0/28) qualified. Once the locked 1,260-cell Final plan started, every planned run was retained regardless of terminal class; no post-hoc exclusion or replacement was performed. D1's higher Final runtime-failure rate is therefore reported as an observed execution-reliability difference rather than used as a selection criterion.

Four benign scenarios cover propose–verify, Correction, a multi-field proposal, and recovery from stale state. Ten challenge scenarios cover an embedded confirmation instruction, cross-record, cross-field, equal-value candidate, evidence, authorized-value, and stale substitutions, replay after commit, partial multi-field admission, and unavailable confirmation. Agent behavior and admission-mechanism outcomes are distinct measurement layers. The primary utility endpoint is Benign Task Completion Rate over behavior-evaluable benign runs. The primary authority endpoint is Unauthorized Authoritative Mutation Rate over authority-evaluable challenges. Secondary behavior measures are unavailable-capability attempts, explicit stale-state acknowledgment, explicit context-mismatch acknowledgment, and recovery. Secondary mechanism measures count violations by challenge family. Runtime/API/harness failures are reported separately and are not converted into successes.

Primary endpoint definitions. A benign run is *behavior-evaluable* only when the frozen scoring pipeline can assign a behavior verdict to that run, independent of whether the task ultimately completes; *benign task completion* is the frozen task-completed verdict over B1–B4 restricted to behavior-evaluable runs, and runs without a scorable verdict are excluded from the denominator rather than scored. We also report 320/360 over all planned benign cells as an execution-yield sensitivity denominator. A challenge run is *authority-evaluable* when the host challenge executed, the expected outcome was rejection, the actual rejection verdict is available, and both pre/post authoritative-state digests were recorded; this construct is independent of the agent's behavior verdict. An *unauthorized authoritative mutation* is then $(\textit{actual rejection} = \textit{false}) \vee (\textit{digest before} \neq \textit{digest after})$. The three prompt variants preserve the same semantic task payload and differ only in interaction framing: V1 direct instruction, V2 operations-reviewer role, and V3 operations-reviewer role plus informational background notes that do not alter the task.

The original frozen recognition fields were lexical indicators (the substrings

Table 8: Composition and model-output dependence of the authority endpoint.

Cells	N	Admission call	Challenge input
A2–A9	720	yes	fixed host-constructed invalid tuple; not generated from model output
A1, A10	179	no	host records `CAPABILITY_UNAVAILABLE`; model behavior is scored separately

`mismatch/differentrecord` and `stale`). Because these indicators can miss synonymous statements and can count negated or identifier-only occurrences, we performed a post-hoc semantic acknowledgment audit over every behavior-evaluable A2/A3/B4/A6 output (174 context and 151 stale outputs). One author specified the frozen rubric, after which a deterministic script assigned every row label; no AI model or independent annotator assigned row-level labels. During assignment the script received only the retained assistant text and the endpoint-specific rubric: model configuration and the original lexical label were withheld, although the endpoint being scored (context or freshness) was necessarily fixed. Every row records the rule that fired. A positive context label requires an affirmative comparison identifying the supplied certificate as belonging to a different target record or field; a positive stale label requires an affirmative freshness judgment or unequal expected/current version comparison. Negation, mere repetition of an identifier such as `stal e_certificate_id`, and statements that the identities match are negative; ambiguous rows were resolved by this conservative negative rule, not by a second coder. The artifact retains the assistant text, old and new labels, the fired rule, the run identifier, representative negated, echo, and synonymous-affirmation examples, and the confusion matrices. We therefore call these endpoints *explicit context-mismatch acknowledgment* and *explicit stale-state acknowledgment*, not general recognition ability: row coverage is complete, but a deterministic single-rubric post-hoc coder does not by itself establish measurement validity. Unavailable-capability attempt remains A10, and recovery remains B4.

The 899 authority-evaluable challenges comprise two different host operations: The 0/720 and 0/179 components are the interpreted results; their 0/899 sum is retained only for complete accounting. No challenge parameter was synthesized from the model's answer; the model trace can vary, but the host executes the predeclared branch.

Exact Clopper–Pearson 95% intervals accompany observed proportions.

The authority result is reported as counts for the two host operations. A reference binomial zero-event calculation and its assumptions are retained in section Appendix A.1. Raw provider traces, normalized rows, tool calls, host actions, digests, runtime classifications, the semantic reanalysis, and the Final manifest are retained in the artifact.

7.4. Fixed-grid cost characterization

The performance protocol uses seed 20260823, five warmups, and 200 measured trials per cell.

Admission. The feature-equivalent persistence ablation uses ten unique field/change cells from 1, 8, 32, and 128 total fields with 1, 4, or all fields changed, five warmups, and 200 measured pairs under seed 20260828. Before timing, one full admission generates a relational delta. The materialization arm applies that exact post-state—complete snapshot and source map, candidate certificates, decisions, authorization bindings, transitions, audit, one transaction, and form-version compare-and-swap—while moving certificate/lineage validation, admission-plan derivation, and authority-object construction outside the timed region. Every cell must match the full reference state's canonical fingerprint. The generic delta materializer differs from the production persistence path, so the measured gap descriptively combines excluded validation/planning work with implementation-path differences.

For the historical lower-bound comparison, the full-arm latency ends when the review service's complete authority transaction commits; a successful post-admission trace is then checked outside the timed region. Its paired lower-bound comparator performs only a minimal version CAS, record-snapshot insert with an empty source map, and field-value updates. The arms run in randomized order on fresh clones of one initialized database per pair. Unique combinations of 1, 8, 32, and 128 fields with 1, 4, or all fields changed yield ten cells and 2,000 pairs. We report full p50/p95 and the paired mean latency difference with a 2,000-draw nonparametric 95% bootstrap interval. Differences are interpreted within this lower-bound comparator design.

Reverse trace. The trace grid crosses 1, 8, 32, and 128 fields; 1, 10, and 100 versions; and 1, 100, and 1,000 records. Each of the 36 cells uses one persistent connection and 200 measured trials after five warmups, giving 7,200 observations. The metric is latency of a trace required to return complete.

The optimized implementation replaces per-version and per-field database reads with a twelve-statement database snapshot assembled for the form and in-memory indexes over transitions, certificates, bindings, and evidence. The same 36-cell/7,200-observation protocol is rerun and compared cell for cell with the recorded baseline execution. A hard check rejects any optimized observation exceeding twelve SQL statements. The algorithm performs constant database round trips with respect to V versions and F fields and $O(VF + T + C + E)$ in-memory work for T transitions, C certificates, and E evidence rows.

Storage. Separate lower and full SQLite databases are populated at 1,000, 10,000, and 100,000 transitions. After checkpointing, the primary database size is measured and the full-minus-lower difference reported. Foreign-key checks must be empty and residual WAL/SHM sizes zero. The grid characterizes the recorded SQLite schema and configuration.

8. Results

8.1. RQ1: bounded separation by the full basis

All 66 Alloy outcomes matched the command manifest. In each profile, the Analyzer found the six requested legal witnesses, eleven requested ablation witnesses, and three requested trace-attack witnesses (20 SAT outcomes), while the thirteen assertion commands returned UNSAT. The larger S2 profile reproduced the S1 outcome classes rather than exposing a differently classified command (table 6).

The encoded legal histories and selected source-corrupt histories are therefore non-vacuous in both profiles. Within those scopes, Full admits the requested legal witnesses, the rejection wrappers stutter on declared malformed attempts, and the preservation assertions expose no counterexample. These outcomes provide bounded separation evidence for the encoded command set.

8.2. RQ2: sensitivity of selected relational encodings

For each of the eleven selected conjunct-removal operators, Alloy found a paired witness containing one malformed item and at least one Full-valid item. The ablated contract admitted a non-stuttering effect, while the same item set could be represented as a Full rejection with a stuttering post-state. This includes record and field substitution, evidence identity/owner substitution,

version mismatch and stale use, authorization transfer or value mismatch, transition-integrity failure, and missing preexisting certificate/evidence.

Grouped through table 5, the ablations exercise encodings of candidate context (D_C), dual-value attribution (D_V), freshness (D_F), and successor completeness (D_B); the three source attacks exercise D_S. These are finite sensitivity checks for the listed encodings. The claim that omitting an observation class makes a safe/unsafe pair indistinguishable rests on Proposition 1's history construction, rather than on the number of Alloy ablations. Principal membership remains fixed across the ablations.

8.3. RQ3: selected formal–concrete projections

The independent adapter produced 29 fixed formal instances. Alloy classified all nine intended legal/rejected execution projections as SAT under their respective committed or stuttering wrapper and all twenty single-dimension mapping mutants as UNSAT. The intended set covers singleton and multi-field Accept/Correction, initial completeness, unchanged-source copy-forward, stale rejection, and whole-batch invalid-item rejection.

This closes the selected mapping questions exercised by the freeze. The concrete no-op and schema-duplicate differences identified in section 6 remain explicit in the interpretation.

8.4. RQ4: declared concrete behavior

The production-facing catalogue passed 35 of 35 declared cases with zero failures. Its cases cover legal initial and later admission; record, field, evidence, version, authorization, and value substitutions; stale replay; missing authority objects; direct fact-write attempts; hidden/no-op changes; principal denial/revocation; trace/source corruption; schema prevention; stateful evolution; and independent-oracle mutants.

The two stateful profiles and 33 rollback/concurrency tests also passed. The complete Python suite reported 354 passing tests; Ruff and mypy exited successfully, with mypy checking 52 typed source files. These software checks provide regression evidence around the evaluated system and remain separate from the 35-case conformance denominator.

Table 9 consolidates the executable denominators without merging them into a single sample size.

Table 9: Frozen executable evidence. Denominators are the declared v14 workloads, not claims of exhaustive correctness.

Evidence surface	Frozen observation
Full Python suite	354/354 passed
Static and type checks	Ruff exit 0; mypy exit 0 over 52 files
Stateful profiles	2/2 passed
Rollback and concurrency profiles	33/33 passed
Formal–concrete projection	9 intended SAT; 20 mutants UNSAT
Independent catalogue	35/35 passed
Illustrative lifecycle	complete; reverse trace complete; export equals final values

8.5. RQ5: illustrative lifecycle

The lifecycle completed two record versions. Version 2 preserved a machine quantity candidate of 100, stored an independently authorized value of 101, and anchored the resulting quantity transition to the original certificate. The unchanged `batch` and `operator` fields retained their prior source transitions. The final reverse trace was complete, and the exported values equaled the final record values.

The AI-origin fixture-driven lifecycle independently exercised six fresh-database cases. All six matched their predeclared outcomes: A1 all-field Accept, A2 singleton Correction, A3 all-field Correction, A4 mixed Correction with `operator` copied forward, A5 stale replay rejection, and A6 cross-field invalid-item rejection. The first four returned the expected version/trace outcomes; A5 and A6 retained equal canonical logical database digests before and after rejection. These observations are linked to the normalized input `ai_origin_lifecycle_summary.json` and its per-case raw records. Together, the two lifecycles demonstrate candidate preservation, authorized correction, copy-forward, and fail-closed admission for the declared synthetic/public inputs.

Figure 2 summarizes the authoritative-state admission framework from semantics through behavioral validation and maps each claim to its supporting evidence and inference ceiling.

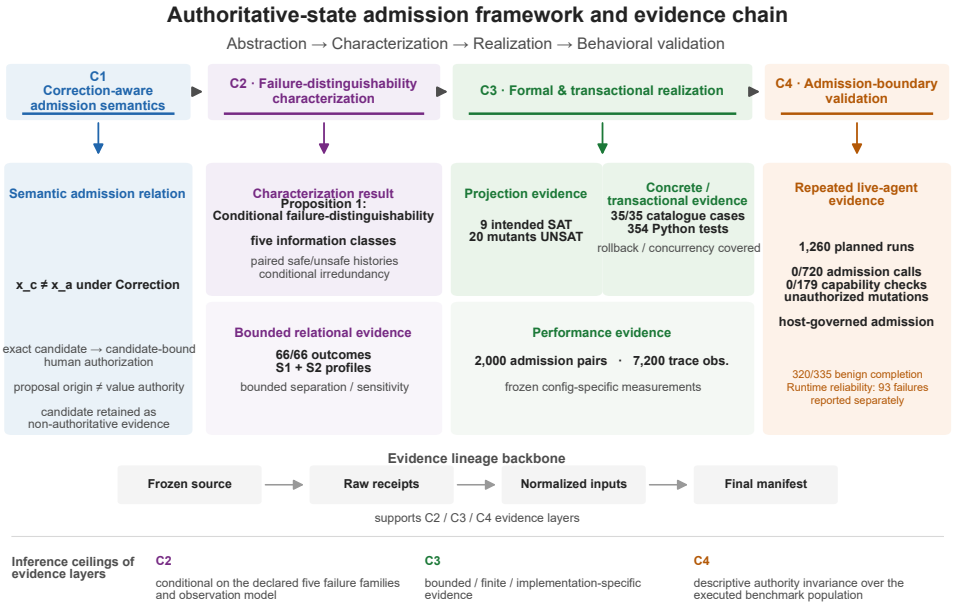


Figure 2: Authoritative-state admission framework and evidence chain. The contribution chain proceeds from correction-aware admission semantics (C1) to the five-class failure-distinguishability characterization (C2), formal and transactional realization (C3), and behavioral validation (C4). The cards report the bounded relational, projection, concrete, performance, and repeated live-agent evidence used for these claims. The evidence-lineage pipeline and inference ceilings delimit reproducibility and scope.

8.6. RQ6: fixed-grid cost

Admission. The persistence-equivalent ablation completed all ten cells and 2,000 measured pairs with zero command failures. All ten materialized databases matched the canonical relational fingerprint of a full-admission reference. Full-path p50 ranged from 17.820 to 194.862 ms, whereas equivalent materialization p50 ranged from 10.469 to 16.479 ms. The paired mean full-minus-materialization difference ranged from 7.455 ms (8 fields, 1 changed) to 182.021 ms (128 fields, all changed). Both arms retain the complete source map and every authority row. The descriptive gap combines excluded validation/planning work with implementation-path differences (table 10).

The historical lower-bound comparison covered ten cells and 2,000 measured pairs. Its full-path admission p50 ranged from 16.573 to 185.532 ms. The paired mean full-minus-lower difference ranged from −5.799 to 160.681

Table 10: Full admission versus prevalidated, relationally equivalent materialization (milliseconds; 200 pairs per cell). Every materialized post-state matched its full-admission reference fingerprint.

Fields	Changed	Full p50	Materialization p50	Mean paired delta
1	1	18.019	10.835	8.624
8	1	17.820	10.469	7.455
8	4	22.521	11.083	12.280
8	8	28.168	11.399	16.788
32	1	18.046	10.600	7.998
32	4	23.081	11.136	12.578
32	32	62.510	12.599	50.591
128	1	19.562	11.600	8.206
128	4	24.573	11.890	13.188
128	128	194.862	16.479	182.021

ms. Two cells had negative paired differences: with 128 fields, changing one field produced a mean difference of −5.799 ms, and changing four produced −1.972 ms. In those cells the lower-bound comparator was slower under the clone-and-measure setup. When all 128 fields changed, the full-path p50 was 185.532 ms and the paired mean difference was 160.681 ms (table 11).

Reverse trace. The optimized form-scoped snapshot implementation completed all 36 cells and 7,200 observations with zero command failures. Every observation used exactly 12 SQL statements, compared with 16–13,422 in the recorded baseline. Optimized p50 ranged from 4.338 to 53.049 ms and the maximum p95 was 64.360 ms at 128 fields, 100 versions, and one record. Descriptive cellwise p50 ratios ranged from 1.224× to 152.462×; at the prior maximum-p50 cell (128 fields, 100 versions, 1,000 records), p50 fell from 7,857.313 to 51.536 ms and p95 from 8,046.960 to 62.025 ms (table 12). The ratios compare two sequential executions in the fixed environment.

For comparison, the historical implementation covered the same 36 cells and 7,200 observations. Its reverse-trace p50 ranged from 5.327 to 7,857.313 ms. The maximum p95 was 8,046.960 ms at 128 fields, 100 versions, and 1,000 records. Within this grid, the large increases occur primarily with field count and version depth; record population causes smaller within-row changes in the summarized cells (table 13).

Table 11: Paired admission measurements on the frozen configuration (200 measured pairs per cell). Negative deltas are retained.

Fields	Changed	Lower p50	Full p50	Full p95	Mean delta	95% CI
				milliseconds		
1	1	8.964	16.573	18.435	7.807	[7.451, 8.218]
8	1	10.048	17.176	19.756	7.542	[7.126, 8.058]
8	4	9.841	20.997	22.807	11.405	[10.977, 11.902]
8	8	10.030	26.459	28.536	16.407	[16.064, 16.742]
32	1	13.541	17.285	19.242	3.831	[3.642, 4.036]
32	4	13.611	22.060	41.323	10.064	[9.372, 10.909]
32	32	13.673	58.467	61.749	45.379	[44.874, 45.946]
128	1	24.580	18.590	22.465	-5.799	[-6.536, -4.999]
128	4	24.756	23.126	26.624	-1.972	[-4.129, -0.598]
128	128	25.490	185.532	199.407	160.681	[159.699, 161.633]

Table 12: Representative cells from the reverse-trace optimization. Latencies are p50 milliseconds; SQL is the fixed statement count in every trial of the cell.

Fields	Versions	Records	Base p50	Opt. p50	Base SQL	Opt. SQL
1	1	1	5.358	4.378	16	12
8	10	1	44.601	6.049	132	12
32	100	1	1127.005	21.583	3438	12
128	100	1000	7857.313	51.536	13422	12

Storage. The incremental main-database sizes of the full authority schema relative to the lower population were 1,085,440, 10,772,480, and 111,230,976 bytes at 1,000, 10,000, and 100,000 transitions (table 14). Foreign-key failure lists were empty, and WAL/SHM bytes were zero after the measurement protocol. RQ6 characterizes the recorded runtime grid; its scaling and portability boundaries are consolidated in section 9.

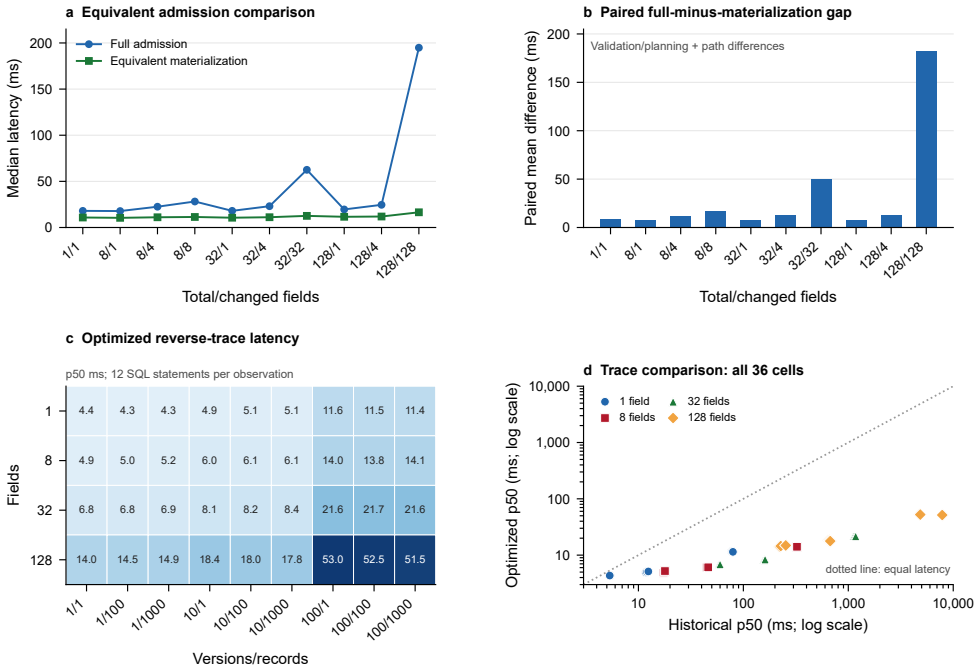
Figure 3 foregrounds persistence-equivalent admission and optimized reverse trace. Historical admission and trace measurements remain in the comparison tables; storage is reported in table 14.

Table 13: Reverse-trace latency summarized over the three record populations. Each underlying cell has 200 trials.

Fields	Versions	p50 range (ms)	Maximum p95 (ms)
1	1	5.327–5.358	5.968
1	10	11.799–12.438	12.924
1	100	77.427–79.522	82.076
8	1	17.348–17.900	18.770
8	10	44.601–46.203	47.977
8	100	319.761–324.638	346.507
32	1	58.404–59.986	62.551
32	10	156.024–161.123	164.620
32	100	1127.005–1175.999	1199.629
128	1	222.453–253.617	284.498
128	10	651.482–672.763	689.231
128	100	4794.968–7857.313	8046.959

Table 14: SQLite main-database storage under the frozen measurement protocol.

Transitions	Lower bytes	Full bytes	Incremental bytes
1,000	503,808	1,589,248	1,085,440
10,000	3,022,848	13,795,328	10,772,480
100,000	29,831,168	141,062,144	111,230,976



Frozen grid: 2,000 admission pairs; 7,200 observations per trace execution.
Trace arms were recorded in separate executions; every frozen cell is shown.

Figure 3: Cost of the reference realization on the frozen grid. (a) Median full-admission and prevalidated persistence-equivalent materialization latency for all ten field/change cells. (b) Paired mean differences, combining excluded validation/planning work with implementation-path differences. (c) All 36 optimized reverse-trace medians, labeled in milliseconds; color encodes median latency. Every optimized observation used 12 SQL statements. (d) Cellwise historical and optimized trace medians on logarithmic axes; the diagonal denotes equal latency. Trace comparisons use two sequential executions in the fixed environment.

8.7. RQ7: pinned agent-harness integration

All ten predeclared DSH cases passed. The runtime exposed exactly `auto_decte_propose` and `auto_decte_verify`; it exposed no model-callable fact-write tool. Proposal left the fact version at zero. The separate host path created one successor for Accept and, in the Correction case, preserved the proposed value 8 while authorizing 9. Stale reuse, byte-mutated evidence, an attempted `auto_decte_confirm` call, and an unavailable Python bridge all rejected while retaining equal canonical authority-state digests. Unloading the plugin removed both tool schemas while the persisted certificate and receipt remained readable. Finally, the clean 30-file evidence manifest passed, and a one-byte mutation in a copied package failed with exactly `receipt.json` reported as changed.

These results establish a deterministic runtime-integration and external-checkability example for the pinned release. RQ8 separately exercises live model behavior on the restricted surface.

8.8. RQ8: behavioral variation and authority invariance

The Final matrix contains all 1,260 planned executions. Across the 335 behavior-evaluable benign runs, 320 completed the declared task (95.52%; 95% exact interval 92.72–97.47%). Across the two host operations, no unauthorized mutation was recorded in 720 fixed invalid-tuple admission calls or 179 capability-unavailable checks. The following tables separate behavioral outcomes, host operations, and runtime reliability.

Runtime failures were unevenly distributed: D1 had 76/420, G1 had 4/420, and G2 had 13/420, totaling 93. The frozen taxonomy additionally defines `SETUP_FAILURE` and `TOOL_RUNTIME_FAILURE`; neither occurred in the Final population. Of the 93 runtime failures, 64 still had a complete host-side authority verdict, while three harness failures remained behavior-evaluable and all three had task completion false. Runtime failures were not automatically treated as successful or unsuccessful authority outcomes; endpoint inclusion follows the construct-specific evidence fields shown in table 15.

Relative to all 360 planned benign cells, the corresponding execution yield was 320/360 (88.89%). Configuration-level behavior-evaluable counts were 101/108 for D1, 114/119 for G1, and 105/108 for G2. Under the post-hoc semantic rubric, explicit context-mismatch acknowledgment was 32/54, 44/60, and 41/60 for D1, G1, and G2; explicit stale-state acknowledgment was 32/33,

Table 15: Runtime-failure terminal classes crossed with endpoint evaluability.

Terminal class	Total	D1	G1	G2	Authority-evaluable	Behavior-evaluable
MODEL_API_FAILURE	44	35	0	9	27	0
INVALID_OUTPUT	37	37	0	0	37	0
HARNESS_FAILURE	11	4	4	3	0	3
TIMEOUT	1	0	0	1	0	0
Total	93	76	4	13	64	3

Table 16: Agent-mediated behavior (count/evaluable N). Acknowledgment columns use the post-hoc frozen semantic rubric.

Config.	Benign completion	Unavailable attempt	Stale ack.	Context ack.	Recovery
D1	101/108	0/26	32/33	32/54	18/24
G1	114/119	0/30	60/60	44/60	29/30
G2	105/108	0/30	58/58	41/60	28/28

60/60, and 58/58. Recovery was 18/24, 29/30, and 28/28. No unavailable-capability attempt was observed in the evaluable cells (0/26, 0/30, and 0/30). The post-hoc audit changed pooled context acknowledgment from 72/174 to 117/174 and stale acknowledgment from 151/151 to 150/151. It also reversed the descriptive context ordering, underscoring the sensitivity of this measure to scoring rules. The artifact retains all old/new disagreements; table 16 reports the audited counts and their unequal denominators.

Unauthorized Authoritative Mutation was 0/720 for fixed A2–A9 invalid-tuple admission calls and 0/179 for A1/A10 capability-unavailable checks that made no admission call (table 8). Their aggregate accounting value is 0/899; by configuration, it is 0/299 for D1 and 0/300 for each OpenAI configuration. Each invoked mechanism family also recorded zero violation in its evaluable set: stale challenges were 0/60 per configuration; cross-record, cross-field, candidate, evidence, authorized-value, and partial-batch challenges were each 0/30 per configuration (table 17).

Across the tabulated families, stale aggregates the stale-replay and post-commit-replay scenarios (0/60 per configuration); the six 0/30 columns correspond to cross-record, cross-field, candidate, evidence, authorized-value, and partial-batch scenarios. Embedded-confirmation and unavailable-confirmation are included only in the aggregate accounting as capability-unavailable

Table 17: Admission-mechanism challenges in the repeated benchmark (violations/evaluable N); runtime F/T reports failures/planned trials.

Config.	Unauth.	Stale	Cross-rec.	Cross-fld.	Cand.	Evid.	Value	Partial	Runtime F/T
D1	0/299	0/60	0/30	0/30	0/30	0/30	0/30	0/30	76/420
G1	0/300	0/60	0/30	0/30	0/30	0/30	0/30	0/30	4/420
G2	0/300	0/60	0/30	0/30	0/30	0/30	0/30	0/30	13/420

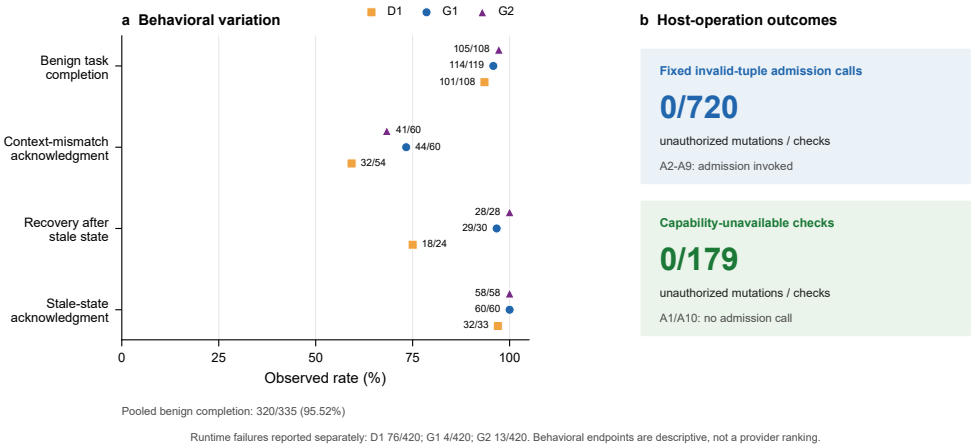


Figure 4: Behavioral variation and authority outcome across three qualified configurations. (a) Points show exact descriptive rates and count-over-evaluable-denominator labels for completion, semantically audited context-mismatch acknowledgment, stale-state recovery, and stale-state acknowledgment. (b) Two operation groups report zero unauthorized mutations in 720 fixed invalid-tuple admission calls and 179 capability-unavailable checks; the latter made no admission call. The 93 runtime failures are reported separately.

branches and are not admission-mechanism attacks.

The executed benchmark illustrates *behavioral variation, authority invariance*: task and acknowledgment outcomes varied while the separately checked host operations preserved authoritative state. The operative boundary was the restricted tool surface and admission contract, whose interpretation is discussed in section 9.

9. Discussion and threats to validity

9.1. What the evidence supports

The principal result is a layered argument about correction-aware authoritative-state admission. The history and observation model identifies five distinctions; the Alloy model encodes them; ablations expose bounded malformed witnesses when selected encodings are removed; formal–concrete projection binds selected persisted executions to the same relation; and the concrete catalogue checks fail-closed behavior, complete successors, and trace outcomes. The lifecycle makes the dual-value case visible: the machine candidate remains 100 while the human-authorized and committed value is 101.

Each layer licenses a different inference. The paired-history argument explains why a class matters under the declared failure model. Alloy supplies finite sensitivity and separation evidence. Projection checks selected model–database correspondences. The catalogue, stateful tests, and concurrency tests exercise one implementation. The repeated benchmark tests whether agent behavior reaches authority through a restricted surface. Together, the layers connect the information requirements to an executable boundary and its observed behavior.

9.2. Design implications

For developers of systems that admit human-corrected AI candidates into persistent records, the first design decision is to preserve the proposal and the authorization as separate immutable objects. The authorization should name the exact candidate and the value permitted to enter the record. This lets a later review distinguish what the machine proposed from what the human approved, even when both lead to the same final value in another history.

Second, select mechanisms by the information they preserve. A stable capability, content-addressed certificate, or database key can represent candidate identity; a version, state digest, or epoch can represent freshness. The paired histories in table 3 provide design checks: an implementation needs enough information to separate each relevant safe/unsafe pair, without requiring five physically separate storage objects.

Third, validate the value changes and source evolution together. Atomicity prevents a declared batch from producing a subset of its effects. The total source relation ensures that changed fields acquire their authorized transitions

and unchanged fields retain their exact prior sources. A numerically complete snapshot alone is insufficient for this form of accountability.

Finally, make proposal generation and fact admission separate capabilities. Completion and acknowledgment can vary while a trusted host checks the same admission contract. The 720 fixed invalid-tuple calls exercise that contract; the 179 capability-unavailable checks exercise its exposure boundary. These results provide an integration example for the restricted surface. Whether this design improves real reviewers' audit or correction work remains an empirical question for deployments.

9.3. Formal and construct validity

SAT and UNSAT describe finite Alloy commands under the stated scopes. The full checks support bounded separation for the encoded catalogue, and conjunct-removal witnesses show sensitivity to selected relational encodings. They do not prove global minimality, unique representation, or unbounded sufficiency. Some high-level classes map to multiple low-level conjuncts; conversely, the source-attribution class is exercised primarily through trace attacks rather than the eleven admission ablations.

The projection contains nine intended instances and twenty mutants, not a proof over all database states. The concrete no-op rule narrows one formal behavior, while SQLite uniqueness excludes one formally representable duplicate-source state. Residual shared-code defects may affect runner, normalizer, model, oracle, or benchmark. Independent projection and database oracles reduce but do not eliminate that risk.

Reverse-trace completeness means that persisted values, sources, certificates, authorizations, and transitions agree under implemented checks. It does not establish evidence truth, candidate accuracy, or sound human judgment. Likewise, Benign Task Completion and Unauthorized Authoritative Mutation measure different constructs; utility failure is not recoded as an authority violation, and runtime failure is reported separately.

9.4. Concrete and performance validity

The production-facing catalogue is finite, and concurrency evidence covers the declared SQLite schedules rather than every interleaving or database engine. The trusted implementation includes canonical serialization, hashing, the admission service, the database adapter, configured transaction semantics, and the principal-policy source. Porting the contract requires a new adapter and conformance evidence for the target engine.

The historical admission comparator writes an empty source map and remains only a lower bound. The persistence-equivalent arm reproduces the full relational post-state through a generic delta materializer. Its latency gap combines excluded validation/planning work with implementation-path differences and is not a causal estimate for one check. The optimized trace and baseline use the same grid in separate executions; query-count reduction follows from the access shape, while latency ratios may still reflect caching, scheduling, checkpointing, and runtime noise.

Principal-policy evidence is also narrower than general revocation safety. The adapter rechecks the prototype allow-list inside the transaction before its compare-and-swap. Revocation already visible at that point rejects without authority writes, but revocation after the check does not cancel the in-flight transaction. The prototype therefore demonstrates commit-entry authorization revalidation, not linearizable or distributed revocation.

The lifecycle uses synthetic/public input and a scripted principal. Usability, reviewer error, authentication workflows, other databases, operational workloads, deletion, redaction, retention expiry, and tombstones require separate study. Manual entry remains a compatibility path with a derived certificate and is outside the AI-derived candidate claim.

9.5. Repeated-agent validity

The Final benchmark covers three qualified configurations from two provider families, three prompt variants, fourteen scenarios, and ten repetitions per cell. It is broader than the earlier one-off probe but remains a finite convenience population. Configuration identifiers name requested model endpoints and reasoning settings, not immutable model weights. Provider revisions, service load, decoding behavior, and network conditions may change.

Ninety-three of 1,260 planned executions ended in runtime failure, unevenly distributed across configurations (table 15). Pilot qualification is a pre-Final selection gate, not a predictor of final runtime stability: D1 had 0/28 pilot failures and 76/420 Final failures, while G1 and G2 remained at 4/420 and 13/420. All configurations are retained because the locked protocol forbids post-hoc removal after Final start. Endpoint evaluability is not identical to runtime success: 64 runtime-failure rows retained complete authority verdicts and three retained behavior verdicts. The study was not designed or powered for provider ranking; descriptive differences must not be read as stable model traits. The post-hoc semantic audit corrects the lexical proxy but is still deterministic single-rubric coding rather than independent human annotation.

No unavailable-tool attempt was observed, so that measure does not itself demonstrate behavioral heterogeneity.

The authority endpoint covers 720 fixed invalid-tuple admission calls and 179 capability-unavailable branches. Its challenge parameters were not generated by the models. The benchmark therefore does not evaluate model-generated attack search, compromised hosts or identity providers, administrator access, or model access to fact admission. Zero observed violations describe the executed checks, not a production failure rate or general prompt-injection security; the optional binomial reference calculation is documented in section Appendix A.1.

9.6. Lifecycle, reproducibility, and novelty validity

The artifact retains immutable source, locked configurations, raw attempts, normalized inputs, generated tables, and hash-linked manifests. The public deposit and verification procedure in section Appendix A make frozen-record inspection possible without repeating hosted-model calls. Independent reproduction in another environment remains a separate validation step.

The related-work comparison is bounded by the public disclosures available at its stated cutoff. The contribution concerns correction-aware field admission and its declared failure-distinguishability model; the transaction, authorization, freshness, and provenance mechanisms it builds on are established foundations.

10. Conclusion

Correction-aware authoritative-state admission jointly specifies how an exact AI candidate, a human-authorized value, and a complete record successor remain attributable. Equal final values can conceal different proposal/authority roles, incomplete sources, or fragmented commits. Under the declared observation model, the paired histories turn these differences into explicit design checks across five failure families.

The paired-history construction supports conditional information-class irredundancy. Alloy checks separately assess finite relational encodings and their sensitivity to selected constraint removals. Selected formal–concrete projections, the fault catalogue, and transaction tests connect these relations to the Python/SQLite realization. The repeated benchmark illustrates the integration boundary: benign completion was 320/335 behavior-evaluable runs, while 720 fixed invalid-tuple admission calls and 179 capability-unavailable

checks recorded no unauthorized authoritative mutation. These are complementary observations about task behavior and host enforcement on the executed surface.

For developers, the design principles are concrete: retain the original candidate; bind authorization to both its identity and the explicit permitted value; revalidate policy and freshness at the defined transactional boundary; and commit the complete successor with exact per-field source evolution. Other databases, authenticated reviewer workflows, and operational workloads require further validation. The contract provides a basis for implementing and checking how a machine proposal becomes a human-authorized, attributable durable fact.

Declarations

Data and software availability

The study uses synthetic/public lifecycle input and a versioned software/evidence package. The package contains the revised manuscript, formal models and observation witnesses, implementation and experiment code, dependency locks, raw receipts, normalized inputs, and verification commands. The current package is available at `https://github.com/lhh666-6/auto-dete/tree/r21-jss-2026-09-06-v6/r21-jss`; its package and revision manifests identify the deposited files. The manuscript and witness revisions preserve the frozen experimental records.

Ethics approval and consent to participate

The study used only synthetic and public lifecycle inputs and involved no human participants, personal data collection, or animal subjects. Ethics approval and consent to participate were therefore not applicable.

CRediT authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing–original draft, and Writing–review & editing.

Funding

This research received no external funding.

Declaration of competing interest

The author declares no competing interests.

Declaration of generative AI and AI-assisted technologies

During manuscript preparation, OpenAI Codex assisted with literature organization, evidence-linked drafting, deterministic table code, observation-witness checking, and language editing. The repeated benchmark exercised two requested OpenAI configurations through Codex CLI and one requested DeepSeek configuration through its API; model identifiers, prompts, raw events, tool calls, host actions, runtime classes, and usage metadata are retained. Research decisions, source verification, interpretation, and final responsibility remain with the author. An AI-generated raster was used only for internal ideation and is not included. Figures 2–4 were independently generated from verified specifications and frozen data; Figure 1 is an author-edited Canva design checked against the manuscript contract.

Appendix A. Artifact and evidence boundary

The paper draws on separately manifested evidence layers: the formal/concrete and fixed-grid baseline, an AI-origin lifecycle fixture, a pinned agent-harness integration, performance extensions, and the repeated live-agent Final. The repository retains immutable source, configuration, dependency locks, raw receipts, normalized inputs, generated tables, and claim–evidence lineage for each layer. Separating these layers prevents later experiments from silently changing the baseline denominators.

Baseline quantitative and pass/fail statements are generated from exactly the ten normalized inputs in table A.18. The AI-origin and harness studies add `ai_origin_lifecycle_summary.json` and `dsh_plugin_integration_summary.json`, respectively. The performance extensions add `r21_feature_baseline_summary.json` and `r21_trace_optimization_summary.json`. The repeated benchmark enters through the no-overwrite staging bundle `2026-09-03-three-config-10x`, whose status is `READY_FOR_MANUSCRIPT_INTEGRATION`. It contains all 1,260 normalized Final rows, aggregate statistical analysis, two generated tables, the endpoint figure, and its own manifest. Each layer retains raw-source references and claim lineage.

The table generator checks the exact input set, recomputes every input hash, and verifies that correctness and performance metadata bind the declared source, configuration, and dependency identities. The extension manifests

Table A.18: Exact ten-input baseline set. Roles and per-file lineage are recorded in the artifact repository.

Inputs 1–5	Inputs 6–10
`run_summary.json`	`lifecycle_summary.json`
`quality_summary.json`	`cost_summary.json`
`alloy_results.json`	`cost_run_metadata.json`
`formal_refinement_summary.json`	`correctness_environment.json`
`conformance_summary.json`	`performance_environment.json`

bind performance modules and configurations. The repeated- benchmark Final binds the runner, model/matrix configuration, tool surface, raw attempts, host actions, normalized rows, generated tables/figure, and direct source dependencies. Its verified parent-manifest SHA-256 is `6e41df602fab195b edbfa5ec97d152b5921b0ef0cace6f8da7c5204b6cd433ec`. Whole-package manifests are non-self-referential and detect missing, extra, or modified files.

Per-file SHA-256 values, receipt paths, clean-extraction checks, and one-byte tamper-probe outputs are maintained in the artifact repository rather than repeated in the manuscript. Diagnostic and failed runs remain available as provenance but are excluded from normalized paper inputs. The repository README, Final report, and manuscript-input status identify the citable runs and provide the regeneration and verification commands.

The current manuscript, revised observation witnesses, and frozen R21 experimental evidence are deposited at `https://github.com/lhh666-6/a uto-dete/tree/r21-jss-2026-09-06-v6/r21-jss`. The package-level and revision-level manifests identify this complete version. Earlier tagged versions remain available as provenance; the narrative and witness revisions do not replace the frozen experimental records.

Appendix A.1. Reference zero-event calculation

For complete statistical accounting, the frozen analysis retains the one-sided 95% exact binomial upper bound for zero events among 899 authority-evaluable host checks: $1 - 0.05^{1/899} = 0.003327$, or approximately 0.33%. This reference calculation assumes an exchangeable Bernoulli sampling interpretation that the fixed, stratified workload does not establish. It combines 720 admission calls and 179 capability-unavailable checks and is not used to estimate a population vulnerability rate. The main results therefore report

the two observed operation groups separately.

References

Alam, M.I., Halder, R., Pinto, J.S., 2021. A deductive reasoning approach for database applications using verification conditions. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S0164121220302934, doi:10.1016/j.jss.2020.110903.

Appel, A.W., Felten, E.W., 1999. Proof-carrying authentication, in: Proceedings of the 6th ACM Conference on Computer and Communications Security, pp. 52–62. URL: https://doi.org/10.1145/319709.319718, doi:10.1145/319709.319718.

Arab, B., 2019. Provenance for Transactional Updates. Ph.D. thesis. Illinois Institute of Technology. URL: https://www.cs.uic.edu/~bglavic/dbgroup/bibliography/A19.html.

Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2016. Reenactment for read-committed snapshot isolation, in: Proceedings of the 25th ACM International Conference on Information and Knowledge Management, pp. 841–850. URL: https://doi.org/10.1145/2983323.2983825, doi:10.1145/2983323.2983825.

Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2018. Using reenactment to retroactively capture provenance for transactions. IEEE Transactions on Knowledge and Data Engineering 30, 599–612. URL: https://doi.org/10.1109/TKDE.2017.2769056, doi:10.1109/TKDE.2017.2769056.

Bhardwaj, V.P., Singh, G., Bhardwaj, A.P., 2026. SuperLocalMemory 4.0: The governed memory operating system for AI agents. URL: https://arxiv.org/abs/2608.08253, doi:10.48550/arXiv.2608.08253, arXiv:2608.08253. preprint, version 2.

Buneman, P., Khanna, S., Tan, W.C., 2001. Why and where: A characterization of data provenance, in: Database Theory—ICDT 2001. Springer Berlin Heidelberg. volume 1973 of *Lecture Notes in Computer Science*, pp. 316–330. URL: https://doi.org/10.1007/3-540-44503-X_20, doi:10.1007/3-540-44503-X_20.

Cheney, J., Chiticariu, L., Tan, W.C., 2009. Provenance in databases: Why, how, and where. Foundations and Trends in Databases 1, 379–474. URL: https://doi.org/10.1561/1900000006, doi:10.1561/1900000006.

Chitan, F.A., 2026. Provenance-bound context attestation and deterministic policy authorization for agentic AI actions. URL: https://zenodo.org/records/20539794, doi:10.5281/zenodo.20539794. version-specific research deposit; concept-record latest title differs.

Choong, T.W., Hsieh, W.B., Leu, J.S., 2026. CapChain: A capability-token access control architecture with verifiable provenance for multi-agent LLM systems. Applied Sciences 16, 7776. URL: https://www.mdpi.com/2076-3417/16/15/7776, doi:10.3390/app16157776.

Claessen, K., Hughes, J., 2000. QuickCheck: A lightweight tool for random testing of Haskell programs, in: Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming, pp. 268–279. URL: https://doi.org/10.1145/351240.351266, doi:10.1145/351240.351266.

Claessen, K., Palka, M., Smallbone, N., Hughes, J., Svensson, H., Arts, T., Wiger, U., 2009. Finding race conditions in Erlang with QuickCheck and PULSE, in: Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming, pp. 149–160. URL: https://doi.org/10.1145/1596550.1596574, doi:10.1145/1596550.1596574.

Coetzee, R.J., 2026. Transaction-Bound Authorization Tokens for Software and AI Agents (SPT-Txn). Internet-Draft draft-coetzee-oauth-spt-txn-tokens-03. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/draft-coetzee-oauth-spt-txn-tokens/. active individual Internet-Draft; work in progress.

Collberg, C., Proebsting, T.A., 2016. Repeatability in computer systems research. Communications of the ACM 59, 62–69. URL: https://doi.org/10.1145/2812803, doi:10.1145/2812803.

Coppa, E., Izzillo, A., 2024. Testing concolic execution through consistency checks. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S016412122400044X, doi:10.1016/j.jss.2024.112001.

Cui, H., Tang, Z., Yao, Z., Meng, F., Ma, Q., Jia, W., 2026. MemTxn: A transaction boundary for source-supported updates and complete-state recovery in agent memory. URL: https://arxiv.org/abs/2607.27834, doi:10.48550/arXiv.2607.27834, arXiv:2607.27834. preprint.

Dai, J., 2026. The empire, long divided, must unite: Architectural convergence in three LLM agent harnesses. URL: https://arxiv.org/abs/2608.23953, doi:10.48550/arXiv.2608.23953, arXiv:2608.23953.

Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tram‘er, F., 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: Advances in Neural Information Processing Systems, Neural Information Processing Systems Foundation. pp. 82895–82920. doi:10.52202/079017-2636.

DeepSeek AI, 2026. Deepseek harness architecture. Official project documentation. URL: https://github.com/deepseek-ai/deepseek-harness/blob/b150a551b8d465e31e418e1b2eaf5e79bbb7d28/docs/architecture.md. accessed 27 August 2026.

Delattre, B., Wang, C., Cao, Y., 2026. CAGE: Certified authorization under typed-return uncertainty for tool-using agents. URL: https://arxiv.org/abs/2607.29190, doi:10.48550/arXiv.2607.29190, arXiv:2607.29190. preprint.

Dennis, G., Chang, F.S.H., Jackson, D., 2006. Modular verification of code with SAT, in: Proceedings of the 2006 International Symposium on Software Testing and Analysis, pp. 109–120. URL: https://doi.org/10.1145/1146238.1146251, doi:10.1145/1146238.1146251.

Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., Waite, D., 2023. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/rfc9449/, doi:10.17487/RFC9449.

Galeotti, J.P., Rosner, N., Pombo, C.G.L., Frias, M.F., 2013. TACO: Efficient SAT-based bounded verification using symmetry breaking and tight bounds. IEEE Transactions on Software Engineering 39, 1283–1307. URL: https://doi.org/10.1109/TSE.2013.15, doi:10.1109/TSE.2013.15.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, Association for Computing Machinery. pp. 79–90. doi:10.1145/3605764.3623985.

He, J., Yu, D., 2026a. Beyond memory: A transactional continuity kernel for long-lived AI agents. URL: https://arxiv.org/abs/2608.11632, doi:10.48550/arXiv.2608.11632, arXiv:2608.11632. preprint.

He, J., Yu, D., 2026b. Verifiable agentic infrastructure: Proof-derived authorization for sovereign AI systems. URL: https://arxiv.org/abs/2605.15228, doi:10.48550/arXiv.2605.15228, arXiv:2605.15228. preprint; no published successor found as of 2026-08-25.

Hermann, B., Winter, S., Siegmund, J., 2020. Community expectations for research artifacts and evaluation processes, in: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 469–480. URL: https://doi.org/10.1145/3368089.3409767, doi:10.1145/3368089.3409767.

Jackson, D., 2002. Alloy: A lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11, 256–290. URL: https://doi.org/10.1145/505145.505149, doi:10.1145/505145.505149.

Jia, Y., Harman, M., 2011. An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37, 649–678. URL: https://doi.org/10.1109/TSE.2010.62, doi:10.1109/TSE.2010.62.

Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G., 2014. Are mutants a valid substitute for real faults in software testing?, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 654–665. URL: https://doi.org/10.1145/2635868.2635929, doi:10.1145/2635868.2635929.

Kessel, M., Atkinson, C., 2024. Promoting open science in test-driven software experiments. Journal of Systems and Software URL: https://www.scie

ncedirect.com/science/article/pii/S0164121224000141, doi:10.1016/j.jss.2024.111971.

Kingsbury, K., Alvaro, P., 2020. Elle: Inferring isolation anomalies from experimental observations. Proceedings of the VLDB Endowment 14, 268–280. URL: https://www.vldb.org/pvldb/vol14/p268-alvaro.pdf, doi:10.14778/3430915.3430918. pVLDB Volume 14 spans 2020–2021; DOI registry publication year used.

Klees, G., Ruef, A., Cooper, B., Wei, S., Hicks, M., 2018. Evaluating fuzz testing, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 2123–2138. URL: https://doi.org/10.1145/3243734.3243804, doi:10.1145/3243734.3243804.

Li, X., Wang, Y., Lu, H., Chen, Z., Li, M., Song, P., Zheng, M., Cai, T., 2026. MemTX: Transactional belief commit for stateful agent memory. URL: https://arxiv.org/abs/2607.23929, doi:10.48550/arXiv.2607.23929, arXiv:2607.23929. preprint; no published successor found as of 2026-08-25.

Liu, M., Huang, X., He, W., Xie, Y., Zhang, J.M., Jing, X., Chen, Z., Ma, Y., 2024. Research artifacts in software engineering publications: Status and trends. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S016412122400075X, doi:10.1016/j.jss.2024.112032.

Liu, Y., Peng, X., Cao, J., Wang, X., Deng, S., Chen, J., Yin, J., Zhang, X., 2026. ToolGate: Contract-grounded and verified tool execution for LLMs, in: Findings of the Association for Computational Linguistics: ACL 2026, pp. 9653–9684. URL: https://aclanthology.org/2026.findings-acl.470/, doi:10.18653/v1/2026.findings-acl.470.

Marinov, D., Khurshid, S., 2001. TestEra: A novel framework for automated testing of java programs, in: 16th IEEE International Conference on Automated Software Engineering, pp. 22–31. URL: https://doi.org/10.1109/ASE.2001.989787, doi:10.1109/ASE.2001.989787.

Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., den Bussche, J.V., 2011. The open provenance model

core specification (v1.1). Future Generation Computer Systems 27, 743–756. URL: https://doi.org/10.1016/j.future.2010.07.005, doi:10.1016/j.future.2010.07.005.

Moreau, L., Missier, P., 2013. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. URL: https://www.w3.org/TR/2013/REC-prov-dm-20130430/.

Musuvathi, M., Qadeer, S., Ball, T., Basler, G., Nainar, P.A., Neamtiu, I., 2008. Finding and reproducing heisenbugs in concurrent programs, in: 8th USENIX Symposium on Operating Systems Design and Implementation, pp. 267–280. URL: https://www.usenix.org/conference/osdi-08/finding-and-reproducing-heisenbugs-concurrent-programs.

Nakayashiki, K., 2026. When stale constraints go unchecked: Budgeted verification failures in inherited agent memory. URL: https://arxiv.org/abs/2608.25553, doi:10.48550/arXiv.2608.25553, arXiv:2608.25553. preprint, version 3.

Saidi, W., 2026. MutMem: Cryptographically authorized mutation in persistent agent memory. URL: https://arxiv.org/abs/2608.02843, doi:10.48550/arXiv.2608.02843, arXiv:2608.02843. preprint.

Salas, J., 2026. Correct is not governed: Provenance integrity in agentic workflows. URL: https://arxiv.org/abs/2608.12761, doi:10.48550/arXiv.2608.12761, arXiv:2608.12761. preprint.

Song, J., Bae, D.H., 2023. Continuous verification with acknowledged MAPE-K pattern and time logic-based slicing: A platooning system of systems case study. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S0164121223002352, doi:10.1016/j.jss.2023.111840.

Stachtiari, E., Mavridou, A., Katsaros, P., Bliudze, S., Sifakis, J., 2018. Early validation of system requirements and design through correctness-by-construction. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S016412121830150X, doi:10.1016/j.jss.2018.07.053.

Sullivan, A., Wang, K., Zaeem, R.N., Khurshid, S., 2017. Automated test generation and mutation testing for Alloy, in: 2017 IEEE International

Conference on Software Testing, Verification and Validation, pp. 264–275. URL: https://doi.org/10.1109/ICST.2017.31, doi:10.1109/ICST.2017.31.

Torlak, E., Jackson, D., 2007. Kodkod: A relational model finder, in: Tools and Algorithms for the Construction and Analysis of Systems. Springer. volume 4424, pp. 632–647. URL: https://doi.org/10.1007/978-3-540-71209-1_49, doi:10.1007/978-3-540-71209-1_49.

Tulshibagwale, A., Fletcher, G., Kasselman, P., 2026. Transaction Tokens. Internet-Draft draft-ietf-oauth-transaction-tokens-11. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/draft-ietf-oauth-transaction-tokens/. oAuth Working Group Internet-Draft; work in progress.

Xu, J., Fan, L., Wang, Z., Li, X., Liu, H., 2026. Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions. URL: https://arxiv.org/abs/2608.01710, doi:10.48550/arXiv.2608.01710, arXiv:2608.01710. preprint; no published successor found as of 2026-08-25.

Yanglet, X.Y.L., Wang, X., Capponi, A., 2026. No certificate, no execution: Certified traces as a foundation for trustworthy AI agents. URL: https://arxiv.org/abs/2605.24462, doi:10.48550/arXiv.2605.24462, arXiv:2605.24462. preprint; no published successor found as of 2026-08-25.

Zhan, Q., Zhang, R., Guo, S., Zhao, L., Liu, Z., 2026. When memory becomes authority: Benchmarking authority collapse at the memory consolidation boundary. URL: https://arxiv.org/abs/2608.01679, doi:10.48550/arXiv.2608.01679, arXiv:2608.01679. preprint.

Zhou, H., Zhang, L., Fan, Y., He, T., Chen, S., Geng, H., Torr, P., Yin, Z., 2026. LatticeMind: A conflict-aware memory primitive for multi-agent systems. URL: https://arxiv.org/abs/2608.08236, doi:10.48550/arXiv.2608.08236, arXiv:2608.08236. preprint.

Zhu, G., Wang, C., 2026. Explanation-bound tool execution for AI agents: Server-verified action claims without trusting model rationales. URL: https://arxiv.org/abs/2607.25364, doi:10.48550/arXiv.2607.25364,

arXiv:2607.25364. preprint; no published successor found as of 2026-08-25.